

THE UNIVERSITY OF DANANG UNIVERSITY OF
SCIENCE AND TECHNOLOGY
FACULTY OF MECHANICAL ENGINEERING

□-----&-----□



**PBL 4: DESIGN OF THE ELECTROMECHANICAL
SYSTEM**

Toppic: Table Tennis Machine

INSTRUCTION: Dr. Pham Anh Duc

STUDENT: Pham Huu Phuc- 1011210087
Le Viet Hoang Linh- 101210319

CLASS: 21CDTCLC1

Da nang, 12th December, 2024

Content

CHAPTER 1: INTRODUCTION	4
1.1 Urgency of the topic	4
1.2 Scientific and practical significance of the topic	4
1.3 Object and scope of research	4
1.3.1 Research object	4
1.3.2 Scope of research	5
1.4 Research methods	5
1.4.1 Methodological basis	5
1.4.2 Specific research methods	5
CHAPTER 2: TOPIC OVERVIEW	6
2.1 General introduction to table tennis	6
2.2 Structure of the table tennis machine	8
2.3 Some table tennis machines in the world	8
2.3.1 Robo Pong 540 Table Tennis Machine (USA)	8
2.3.2 Hiepasac table tennis training machine (Vietnam)	10
2.4 Purpose of the research topic	11
2.5 Software used.....	11
2.5.1 STM32 CubeIDE	11
2.5.2 MATLAB	12
CHAPTER 3: THEORETICAL BASIS	13
3.1 Spin Principle in Table Tennis	13
3.2 Oblique throw motion	14
3.3 Ball Launch Mechanism:	18
3.4 Angle Adjustment Mechanism:	18
3.5 Spin Creation Mechanism:	18
3.6. Ball Push Mechanism:	19
CHAPTER 4: MOTOR SELECTION AND MOTOR MODULE	20
4.1 Firing mechanism	20
4.2 Basketball basket structure	22
4.3 Ball pushing mechanism	24

4.4 Shooting angle rotation mechanism	27
4.5 Firing direction rotation mechanism	29
CHAPTER 5: MECHANICAL DESIGN	33
5.1 Ball reversal mechanism	34
5.2 Ball pushing mechanism	34
5.3 Ball shooting mechanism	35
5.4 Machine operating principle	36
5.5 Technical drawing	36
5.6 Real-Life Model.....	46
CHAPTER 6: ELECTRICAL SYSTEM	47
6.1 Components Used in Electrical Systems	47
6.1.1 DC Motor Driver Circuit - XY-160D	47
6.1.2 L298 DC Motor Driver	49
6.1.3 Mini PWM DC Motor Speed Controller 10A Circuit	50
6.1.4 Stepper Motor Driver Microstep 3.5A 40VDC TB6600	51
CHAPTER 7: APPLYING MATLAB TO SIMULATE TRAVELS, CALCULATE RELATED PARAMETERS.....	54
7.1 Simulation for Obtaining Data for Motor Control Coding.....	54
CHAPTER 8: PROGRAMMING AND CONTROL	72
CHAPTER 9: CONCLUSION AND DEVELOPMENT DIRECTION	96
9.1 Improving Mechanical Design	96
9.2 Using Appropriate Frequencies to Reduce Motor Noise.....	96
9.3 Expanding Operational Modes.....	97
REFERENCES	98

CHAPTER 1: INTRODUCTION

1.1 Urgency of the topic .

Nowadays, with the advancement of science and technology in all fields, especially in the field of research and manufacturing of products to improve human health in general and sports training support equipment in particular. Each sport has exercise machines or support tools to help players practice effectively. And with sports such as table tennis, tennis, badminton..., these are all competitive sports. To be able to practice these sports well, players need to have an opponent of equal strength or stronger to be able to progress. In particular, table tennis is a sport that many people are interested in practicing, from children, teenagers, middle-aged people to the elderly can also play this sport to improve their health. Currently in Vietnam, for those who are new to table tennis or passionate about playing this sport, it is difficult to find someone to practice with or want to improve their level. So the requirement is to have a device that helps players and students to practice by themselves according to pre-set exercises or exercises compiled by coaches suitable for each student. Thanks to that, coaches can easily cover the techniques of all students in order to make timely corrections.

1.2 . Scientific and practical significance of the topic

- Applying one's own scientific knowledge to achieve goals and applying research results to help the community and society.
- Help table tennis enthusiasts have the opportunity to practice and improve their skills, techniques and fitness.
- Contribute research to the training and development of table tennis in Vietnam.

1.3 . Object and scope of research

1.3.1 . Research object

- STM32F103C8T6 microcontroller .
- Matlab analysis of ball position control
- Stepper motor, DC motor, DC reducer
- DC motor driver circuit L298, stepper motor driver TP6600.
- Fusion 360 engineering design software

1.3.2 . Scope of research

- Research and design a machine to practice shooting balls in predetermined positions, controlled by computer.

- Design sample exercises.
- The exercise is performed in an indoor, windless training environment.
- Time to complete the exercise machine is ... months.

1.4 Research methods

1.4.1 Methodological basis

- Consider the relationship between distance, height, shooting angle and shooting speed, using the relationship experimental method because the proposed goal is the influence of one quantity on the other.
- To find the exact shooting angle for a table tennis ball, which is not a point material, cannot be theorized because of the impact of many factors such as shape, resistance, etc. Therefore, the chosen method is to use actual results to adjust according to the requirements.

1.4.2 . Specific research methods

- Study documents: study documents related to DC motors, stepper motors, RC servos, documents related to programming, datasheets of ICs or circuits used.
- Refer to many different sources, including: previous course projects, foreign documents, documents on the Internet, and direct consultation with specialized lecturers of the Department of Mechatronics.
Observation method: based on the phenomena and graph distortions from the engine, the phenomena of starting and exercising the machine to draw conclusions for improvement or find errors.
- Experimental method, trial and error: through testing on test board, debugging, determining the shooting angle and shooting speed to the required points on the table.
- Analytical method: based on experimental results, compare with the requirements and conclude what problems have been solved, what problems have not been solved and how to overcome them.
- Theoretical design and calculation method, applied experimental testing: calculate and select the mechanism based on the requirements, provide design parameters and use Fusion 360 software to design and export processing drawings.

- Planning method: make a specific plan for each job, set small goals to be completed and how long it will take to complete, plan backup plans, give time for each plan, report progress, difficulties and continuously...

CHAPTER 2: TOPIC OVERVIEW

2.1 General introduction to table tennis

Table tennis originated in England, where it was an after-dinner pastime for the upper class during the Victorian era of the 1880s. Table tennis began to develop and become popular in 1901 when table tennis tournaments were held, books about table tennis began to appear, and an unofficial world championship was held in 1902. In 1921 the Table Tennis Association was founded in England, and the International Table Tennis Federation (ITTF) followed in 1926. London hosted the first world championship in 1927. Table tennis was officially introduced as an Olympic sport in 1988.



Figure 2.1: Table tennis competition at the 2012 Olympics

Table tennis is a racket sport played by two players (singles) or two pairs of players (doubles) on a rectangular table divided by a net. Players score points by using their rackets to send the ball over the net and into the opponent's court without the opponent returning it or making a mistake. Each side has only one touch to send the ball to the other side. The winner is the first player to reach 11 points.



Figure 2.2: Standard table tennis table size

Characteristics of a table tennis machine: A table tennis machine is a machine that can shoot the ball to one or more positions on the field similar to a real player. The requirement of the machine is to shoot the ball continuously and accurately, and can adjust many shooting positions, shooting angles, shooting speeds, etc. In addition, the design of the machine must be compact and flexible so that it can be placed or mounted on the table.



Figure 2.3: Practice with the table tennis machine

2.2. Structure of the table tennis machine

The machine consists of 4 parts: ball feeding mechanism, shooting angle adjustment mechanism, ball shooting mechanism and machine body.

- The ball feeding mechanism is responsible for lifting the ball from the ball container to the shooting motor.
- The angle adjustment mechanism helps the motor shoot the ball to different positions. on the table tennis.
- The ball shooting mechanism shoots the ball to specific positions.
- The body connects the parts and ensures the rigidity and stability of the system.

2.3. Some table tennis machines in the world

2.3.1 Robo Pong 540 Table Tennis Machine (USA)

The Newgy Robo Pong 540 table tennis machine has a simple, compact design, can shoot fast, slow, near and far within a straight line and can adjust the spin to 36 angles so that athletes can react to their opponents' spin attacks.



Figure 2.4: Robo Pong 540 with compact design

Machine specifications:

- Maximum number of balls: 86 balls.
- Oscillation angle: 0
- Controller mode: speed, ball frequency.
- Maximum number of balls fired: 63 balls/minute.
- Minimum number of balls fired: 13 balls/minute.
- Advantage:
 - Simple, compact, easy to move.
 - Stable and precise operation.
- Disadvantages:
 - Small number of balls.
 - Simple control mode.

2.3.2 Hiepasc table tennis training machine (Vietnam)

Hiepasc table tennis training machine was manufactured by Mr. Nguyen Trong Hiep, using easy-to-find materials and details with a design that is not too complicated complicated so that those who love this sport can make it themselves to use with the author's guidance.



Figure 2.5: Hiepasc table tennis machine

Hiepasc table tennis training machine was manufactured by Mr. Nguyen Trong Hiep, using easy-to-find materials and details with a design that is not too complicated complicated so that those who love this sport can make it themselves to use with the author's guidance.

Machine specifications:

- Maximum number of balls: 80 balls.
- Ball frequency: 30 - 50 balls/minute.
- Control modes: ball frequency, shot direction, ball spin angle.
 - Advantage:
 - Simple design so anyone can make it themselves.
 - Various control modes.

- Disadvantages:
 - Small number of balls.
 - There is no automatic mode and manual control.

2.4 . Purpose of the research topic

- Manufacture a table tennis shooting machine with a simple, automatic structure, easy to use and diverse control modes, helping players to practice in the most comfortable way.
- Apply the knowledge learned into practice.

2.5. Software used

2.5.1 STM32 CubeIDE

- STM32CubeIDE is an all-in-one multi-OS development tool, which is part of the STM32Cube software ecosystem.



Figure 2.6: STM32CubeIDE software

- STM32CubeIDE is an integrated development environment (IDE) for STM32 microcontrollers, provided by STMicroelectronics. It combines the features of STM32CubeMX (for peripheral configuration and initialization code generation) with a powerful IDE based on Eclipse. STM32CubeIDE supports C/C++ development and offers debugging, compiling, and flashing tools for STM32 microcontrollers. It also provides easy integration with STM32Cube libraries, enabling rapid development of embedded applications for a wide range of STM32

devices. The IDE simplifies development, reduces setup time, and improves productivity for embedded system developers..

2.5.2 MATLAB

- MATLAB (Matrix Laboratory) is a high-level programming language and environment primarily used for numerical computing, data analysis, and visualization. It is widely used in engineering, science, and economics for tasks such as algorithm development, data modeling, simulations, and system analysis. MATLAB provides a rich set of built-in functions for matrix operations, data visualization, and signal processing. Its interactive environment, combined with powerful toolboxes, allows users to easily develop complex computational models and perform data analysis efficiently.

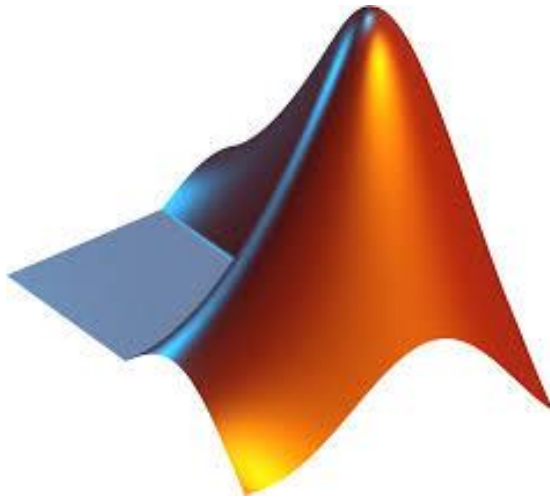


Figure 2.7: STM32CubeIDE software

CHAPTER 3: THEORETICAL BASIS

3.1 Spin Principle in Table Tennis

- At the moment when the racket face contacts the ball, if the direction of the racket's stroke goes right through the center of the ball, the racket will exert a force F on the ball, causing the ball to be hit without spin. If the direction of the racket's stroke does not go right through the center of the ball, in addition to the force F on the ball, it will also create a frictional force with the ball, causing the ball to have spin when bouncing off the racket .
- Main types of vortex :
 - + Top spin: When the racket contacts the ball, in addition to the forward force, there is also the frictional force of the racket with the ball in an upward direction, causing the ball to spin upward.
 - + Downward spin: When the racket contacts the ball, in addition to the forward force, there is also the frictional force of the racket with the ball in a downward direction, causing the ball to spin downward.
 - + Side spin (side spin): When the racket contacts the ball, in addition to the forward force, there is also the frictional force of the racket with the ball in the left or right direction, causing the ball to spin to the side when flying. In which, the frictional force generated between the racket and the ball in the left direction will cause the ball to spin to the left and vice versa.
- Properties of spin ball:

With spin, the ball will bounce on the table and back from the opponent's racket face differently than without spin.

 - + Spin characteristics of the spin ball:

When the ball spins up while flying, the direction of the pressure on the side is downward, so the ball has the characteristics of increasing the speed of falling, increasing the curvature of the arc, shortening the distance of the ball, and reducing the time the ball flies in the air. Therefore, the speed of the ball spins up quickly, the curvature is large, and the rate of good balls is high.
 - + Flight characteristics of a downspin ball:

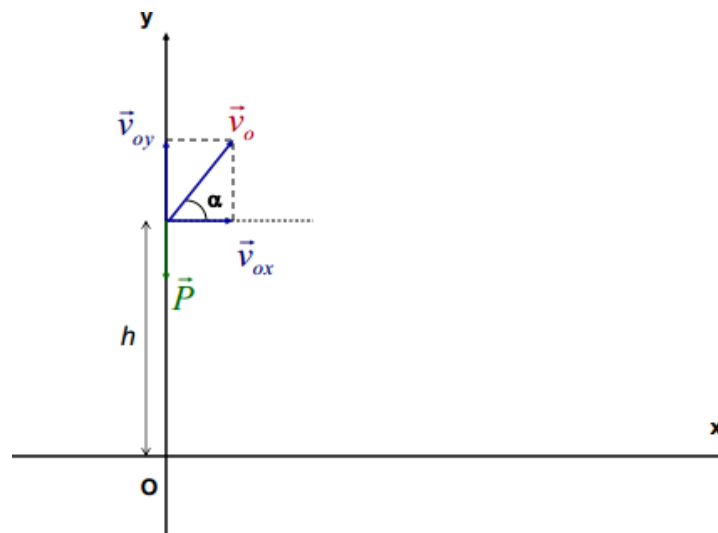
When the ball flies down, the side pressure of the air flow is in the upward direction. Therefore, the flight characteristics of this type of ball are to reduce the falling speed of the ball, reduce the arc, prolong the flight time of the ball, and increase the distance of the ball. Because the arc of the ball with downspin is not good, when the speed is fast, it is easy to fly out of the table, but the ball with downspin at slow speed is relatively stable and solid.

+ Flight characteristics of the side spin ball:

The side pressure of the air flow when the ball spins to the left will move to the right and vice versa.

3.2 Oblique throw motion

- Transfer dynamic throw slant from degree High h: Throwing motion is the motion of an object with initial velocity V_0 and is only affected by gravity (ignoring air resistance).
- Trajectory of the thrown object: The motion of the thrown object is considered as the combination of two motions in the Ox and Oy directions.



- In the Ox direction:

$$x_0 = 0$$

$$v_{0x} = v_0 \cos \alpha$$

$$F_x = P_x = 0 \Rightarrow a_x = 0$$

$$v_x = v_0 \cos \alpha$$

$$x = (v_0 \cos \alpha) \cdot t \quad (1)$$

□ Uniform linear motion.

According to Oy direction:

$$y_0 = h$$

$$v_{0y} = v_0 \sin \alpha$$

$$F_y = P_y = -mg \Rightarrow a_y = -g$$

$$v_y = v_0 \sin \alpha - gt$$

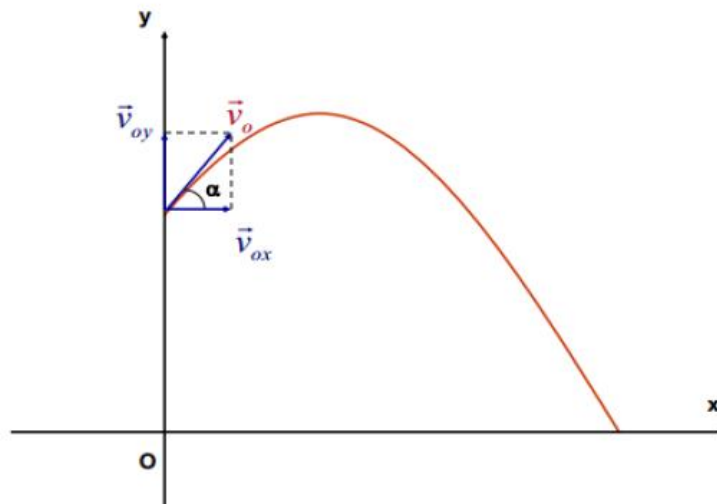
$$y = h + (v_0 \sin \alpha) \cdot t - (gt^2)/2 \quad (2)$$

□ Uniformly variable linear motion.

Substitute t from (1) into (2), we get:

$$y = \frac{-gx^2}{2v_0^2 (\cos \alpha)^2} + (\tan \alpha) x + h \quad (3)$$

□ The trajectory of the object is a parabola.



Based on equation (3), we have:

$$v_0 = \sqrt{\frac{gx^2}{2 \cos^2 \alpha \cdot ((\tan \alpha)x + h - y)}}$$

+) High flight range

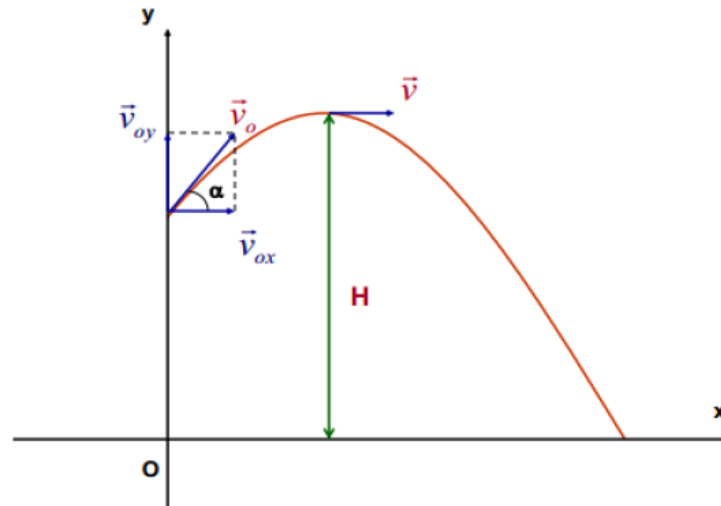


Figure 3.1 High flight range

When the object reaches its maximum height, $v_y = 0$

We have : $v_y = v_0 \sin \alpha - gt$

$$\Rightarrow t = \frac{v_0 \sin \alpha}{g}$$

Substitute t into (2); $y = h + (v_0 \sin \alpha)t - \frac{gt^2}{2}$, we get:

$$H = \frac{v_0^2 (\sin \alpha)^2}{2g} + h = y_{\max} \quad (5)$$

➔ Comment: When v_0 and α increase, H increases, $H_{\max}$ when $\alpha = 90^\circ$

+) Long flight range

Distance between the throwing point and the falling point (both on the ground).

Object hits the ground: $y = 0$

Solve equation (3): $y = \frac{-gx^2}{2v_0^2 (\cos \alpha)^2} + (\tan \alpha)x + h$, I got :

$$L = \frac{v_0^2 (\sin 2a)}{2g} + \frac{V_0 \cos a}{g} + \sqrt{v_0^2 (\sin \alpha)^2 + 2gh} = x_{\max}$$

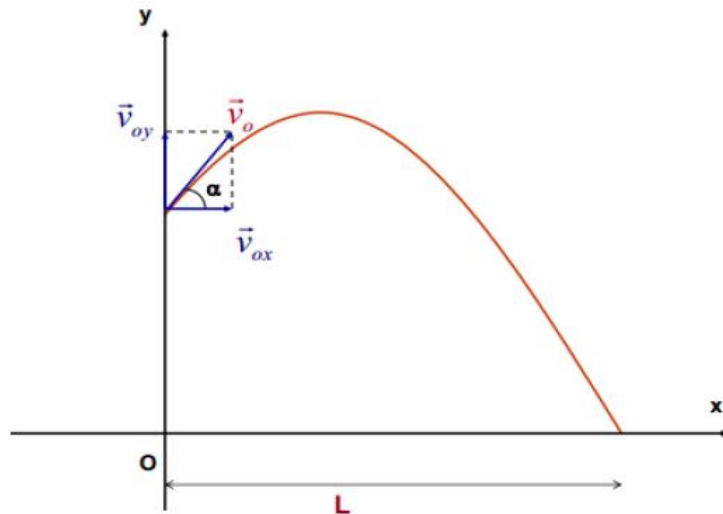


Figure 3.2 Long range

+) Relationship between high and long range flight

$$\tan a = 2 \cdot \frac{(y_{\max} - h) \sqrt{(y_{\max} - h)^2 + h (y_{\max} - h)}}{x_{\max}}$$

Based on the formula:

+) When $y_{\max}$ is constant and $x_{\max}$ is increased:

- The alpha angle decreases.
- The initial firing velocity V_0 increases.

+) When keeping $x_{\max}$ the same and increasing $y_{\max}$:

- The alpha angle increases.
- The initial firing velocity V_0 decline.

➔ Based on that, during the experiment, to shoot to the required points, we can change the parameters accordingly.

3.3 Ball Launch Mechanism:

The launch mechanism uses two motors to generate friction, similar to the way a racket applies force to a ball. The **spinning motor** rotates quickly to create a force that presses and pushes the ball out. The motor marked with "down spin" serves to counteract the ball's spin, providing resistance while adding spin and controlling the force applied.

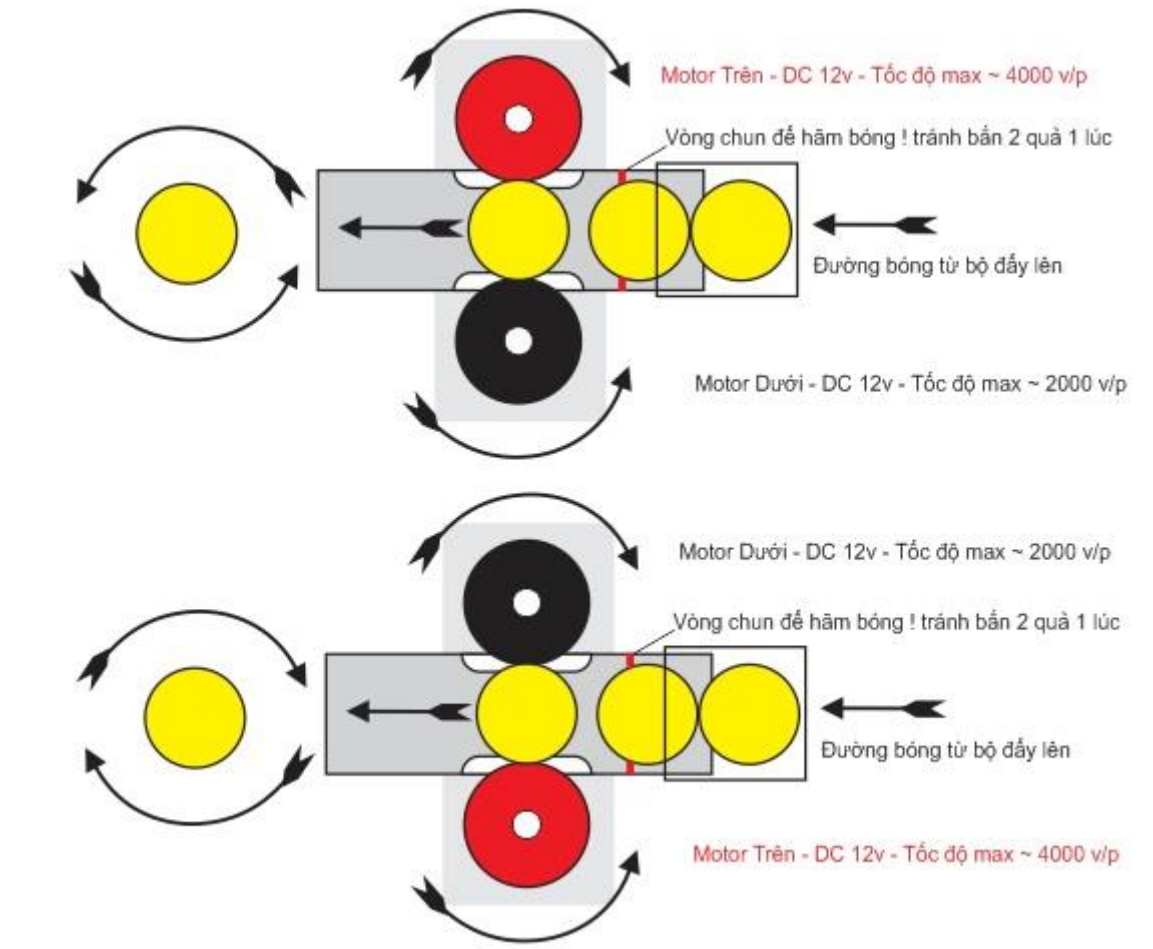


Figure 3.3: Ball Launch

3.4 Angle Adjustment Mechanism:

To change the ball's trajectory, whether high or low, this mechanism also alters the landing point of the ball. By adjusting the angle of the launching mechanism, you can control the ball's path, ensuring that it reaches the desired spot with the appropriate height or angle.

3.5 Spin Creation Mechanism:

A **fast-spinning motor** (marked with "spin" in red) is used to generate spin on the ball. This motor works by creating friction as it interacts with the ball, generating different spin types

such as side spin, topspin, and backspin. Additionally, a motor with adjustable speed (marked with "down spin" in green) is used to control the spin. This motor regulates the ball's spin by slowing it down and increasing the force applied. When running at higher speeds, the ball will travel faster, with less spin, resembling a fast, flat shot.

3.6 Ball Push Mechanism:

The ball is fed into the machine by a mechanism that rotates the ball into a feeding hole. Once the ball has dropped into the hole, the pushing mechanism moves it toward the firing mechanism. This process ensures that the ball is continuously fed and positioned for launch at the correct moment.

CHAPTER 4: MOTOR SELECTION AND MOTOR MODULE

4.1 Firing mechanism

Question: If the frequency of the balls being fired is 8 balls/minute, then how many revolutions/minute will we need the motor to rotate at?

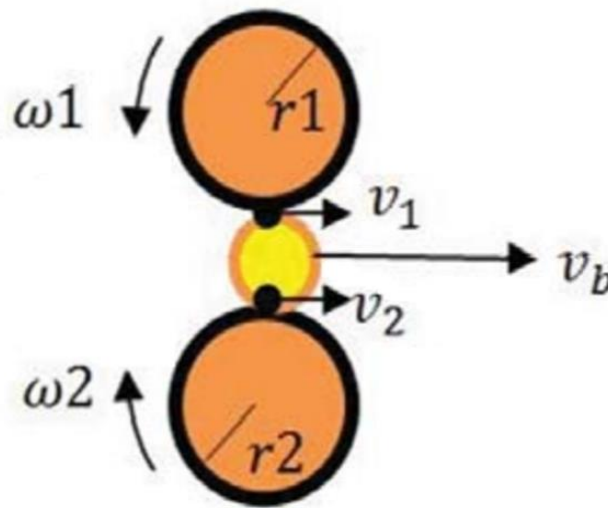


Figure 4.1 Contact pattern between two wheels

v_1, v_2 are the speeds of engine 1, engine 2 (m/s)

v_b is the shooting velocity of the ball (m/s), $4 \leq v_b \leq 10$

w_1, w_2 are the original velocities of engine 1, engine 2 (rad/s)

r_1, r_2 are the radii of rubber wheel 1, rubber wheel 2 (mm)

- Choose $v_b = 5$ (m/s)

- Suppose $v_1 = v_2 = v_b = 5$ (m/s)

Since this is circular motion, we have the formula.

$$v_1 = w_1 \cdot r_1$$

$$\Rightarrow w_1 = v_1 / r_1 = 5 \cdot 1000 / 35 = 225 \text{ (rads)}$$

$$\text{With } w_1 = 2\pi / T_1$$

In which: T_1 is the period (s)

$$\Rightarrow T_1 = 2\pi / w_1 = 2\pi / 225 = 0.028 \text{ (s)}$$

$$\Rightarrow f_1 = 1 / T_1 = 1 / 0.028 = 35.7 \text{ (revolutions/s)} = 2142 \text{ (revolutions/minute)}.$$

- $v_1 = v_2 \Rightarrow f_1 = f_2 = 2142 \text{ (rpm)}$
- With the requirements of the problem and the usage needs, we will choose a motor that can change speed from 2000 (rpm) to 3000 (rpm). Therefore, we can choose the motor as follows.

RS-775 DC Motor

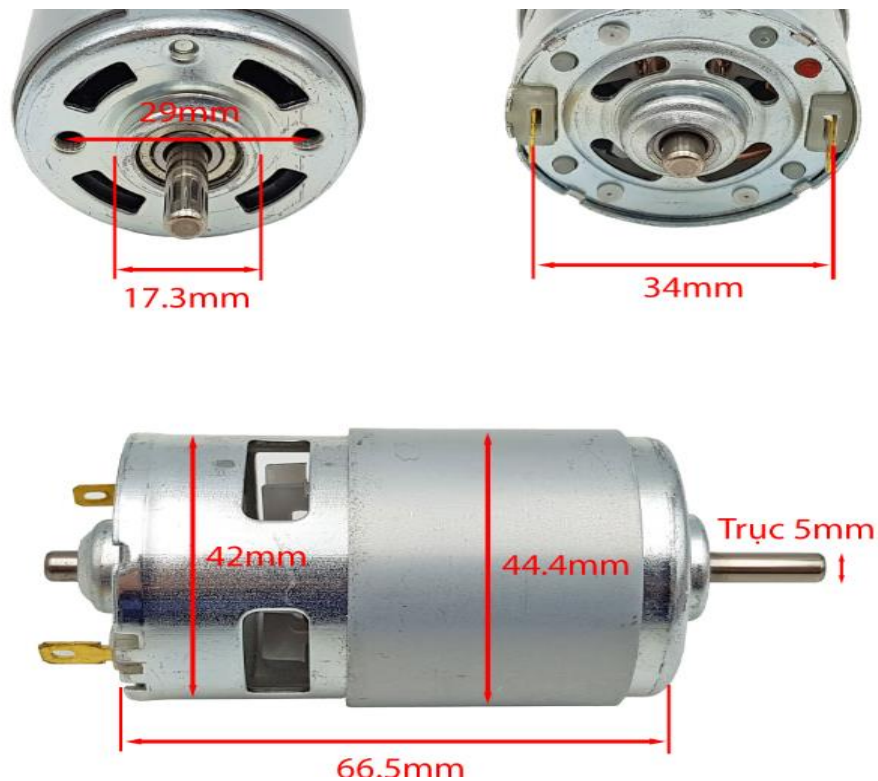


Figure 4.2 RS-775 DC Motor

The RS-775 is larger in size, has more torque and is suitable for applications requiring higher power and torque.

- Specifications
 - * Operating voltage: 12-24v
 - * No-load speed:
 - 12V: About 5000 - 8000 RPM
 - 24V: About 10,000 - 15,000 RPM

*Maximum torque: 1.5-5Nm (12v)

* No-load current : 1.2A-2A

4.2 Basketball basket structure

Input data:

- Basketball basket weight: 300g (0.3 kg).
- Basketball basket diameter: 15 cm (0.15 m).
- Motor used: GA12-N20 Micro Gear Motor (6V, 200 RPM).

Step 1: Calculate the force acting on the basketball basket

Impact force (F):

The force acting on the basketball basket is its gravity, calculated by the formula:

$$F=m \times g$$

In there:

- $m=0.3$ (weight of basketball basket),
- $g=9.81 \text{ m/s}^2$ (gravitational acceleration).

Calculate the impact force:

$$F=0.3 \times 9.81 = 2.943 \text{ N}$$

Step 2: Calculate the required torque

Basketball basket radius:

The basketball basket radius is $r=0.075 \text{ m}$

Torque (T):

$$T=F \times r$$

Substitute the value $F=2.943 \text{ N}$ and $r=0.075 \text{ m}$:

$$T=2.943 \times 0.075 = 0.220725 \text{ N.m}$$

Step 3: Choose the right engine

GA12-N20 engine has parameters:

- Maximum torque: 0.7 N.m.

With the required torque being 0.220725 N.m, we see that the GA12-N20 motor can still meet this requirement, because the maximum torque of the motor is 0.7 N.m, which is much larger than the required torque.

Step 4: Calculate the power required for the engine

The required power (P) is calculated using the formula:

$$P = T \times \omega$$

In there:

- $T = 0.220725 \text{ N} \cdot \text{m}$
- ω is the angular speed (rad/s), calculated from the engine RPM.

Angular velocity ω is calculated by:

$$\omega = 2\pi \times \text{RPM} / 60$$

With RPM = 200:

$$\omega = 2\pi \times 200 / 60 \approx 20.94 \text{ rad/s}$$

So the required power is:

$$P = 0.220725 \times 20.94 \approx 4.62 \text{ W}$$

Step 5: Conclusion

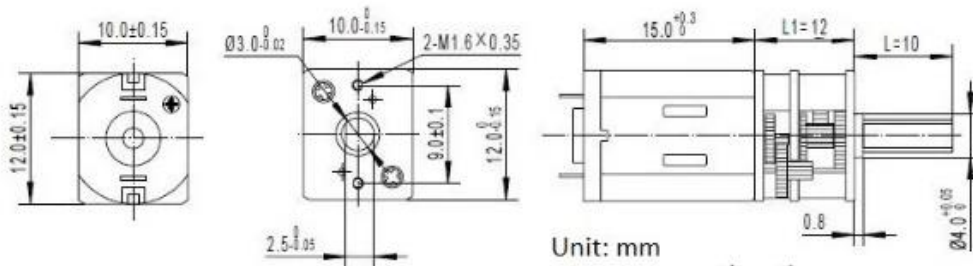
- Torque required to rotate the basket: 0.2207 N.m.
- The GA12-N20 motor can provide a maximum torque of 0.7 N.m, which meets this requirement.
- Power required for the motor: about 4.62 W, the GA12-N20 motor will have enough power to rotate the basket.



Figure 4.3 GA12-N20 motor

Technical specifications table of the motor with gear ratio 1:30 / 1:50 / 1:100 / 1:150, the actual speed will be between the no-load speed and the load speed:

Điện áp (V)	Tốc độ không tải (RPM)	Tốc độ có tải (RPM)	Lực kéo moment định mức (KG.CM)	Dòng điện không tải (mA)	Lực kéo moment tối đa (KG.CM)	Dòng điện chạy được tối đa (mA)	Tỉ số truyền (1:00)
3	250	200	0.07	80	0.56	100	30
3	150	100	0.10	80	0.80	100	50
3	75	60	0.15	80	1.20	100	100
3	50	40	0.20	60	1.60	100	150
6	500	400	0.15	160	1.20	100	30
6	300	240	0.20	160	1.60	100	50
6	150	120	0.30	160	2.40	100	100
6	100	80	0.40	160	3.20	200	150
12	1000	800	0.30	300	2.40	300	30
12	600	480	0.40	300	3.20	300	50
12	300	240	0.50	300	4.00	300	100
12	200	160	1.00	300	7.00	300	150



4.3 Ball pushing mechanism

New input data:

- Table tennis weight: 2.4 g = 0.0024 kg
- Rotation radius: 2 cm = 0.02 m

Step 1: Calculate the force acting on the ball

The force required to push the ball (F) can be calculated using the gravity formula:

$$F = m \times g$$

In there:

- $m = 0.0024$ kg is the weight of the ball,

- $g=9.81 \text{ m/s}^2$ is the acceleration of gravity.

Calculate the impact force:

$$F=0.0024 \times 9.81=0.023544 \text{ N}$$

Step 2: Calculate the required torque

The torque (T) required to rotate the ball pusher can be calculated using the following formula:

$$T=F \times r$$

In there:

- $F=0.023544 \text{ N}$ is the force acting on the ball,
- $r=0.02 \text{ m}$ is the radius of the rotor.

Calculate torque:

$$T=0.023544 \times 0.02=0.00047088 \text{ N} \cdot \text{m}$$

Step 3: Determine the right engine

The RS385 DC motor has a maximum torque of 0.5 N.m , while the calculated torque for the ball pusher is 0.00047088 N.m , which is very small compared to the maximum torque of the motor. This means that the RS385 motor can fully meet the torque requirement of the ball pusher.

Step 4: Calculate the power required for the engine

The required power (P) can be calculated from the torque and angular speed of the motor. The formula for calculating power is:

$$P=T \times \omega$$

In there:

- $T=0.00047088 \text{ N} \cdot \text{m}$ is the required torque,
- ω is the angular speed (rad/s), calculated from the engine RPM.

Angular speed ω calculated from engine RPM:

$$\omega=2\pi \times \text{RPM}/60$$

With $\text{RPM} = 8,000$:

$$\omega = 2\pi \times 800060 \approx 837.76 \text{ rad/s}$$

Calculate required power:

$$P = 0.00047088 \times 837.76 \approx 0.394 \text{ W}$$

Step 5: Conclusion

- Torque required to push the ball: 0.00047088 N.m.
- The maximum torque of the RS385 DC motor is 0.5 N.m, which far exceeds the requirement.
- Power required for the motor: about 0.394 W, while the RS385 motor has a power of ~40 W, powerful enough to push the table tennis ball.



Figure 4.4 RS385 DC motor

RS385 DC Motor is a high speed motor, a device widely used in many fields such as power electronics, applied electronics... It has the effect of converting electrical energy into mechanical energy. RS385 DC motor has fast, strong and stable rotation speed.

- Rated voltage DC 14.4V
- No-load current 0.3A
- No-load speed 14500 rpm
- Suitable voltage 12-15V (12000-15000 rpm)
- Weight 70 grams
- Shaft diameter: 2.3mm

4.4 Shooting angle rotation mechanism

Input data:

- Radius of rotating mechanism: 10 cm = 0.1 m
- Mini stepper motor torque 57HS56: 1.2 N.m
- Weight of rotating mechanism) 1 kg (including 2 DC775 motors and barrel)

Step 1: Calculate the force acting on the rotating mechanism

The force F acting on the mechanism can be calculated from the gravity formula:

$$F=m \times g$$

In there:

- m is the weight of the structure (assuming 1 kg),
- $g=9.81 \text{ m/s}^2$ is the acceleration of gravity.

Calculate the impact force:

$$F=1 \times 9.81=9.81 \text{ N}$$

Step 2: Calculate the required torque

The torque T required to rotate the mechanism can be calculated from the formula:

$$T=F \times r$$

In there:

- $F=9.81 \text{ N}$ is the force acting on the structure,

- $r = 0.1 \text{ m}$ is the radius of the rotating mechanism (because you changed it from 15 cm to 10 cm).

Calculate the required torque:

$$T = 9.81 \times 0.1 = 0.981 \text{ N} \cdot \text{m}$$

Step 3: Compare torque

- Torque required to rotate the mechanism: 0.981 N.m.
- Maximum torque of 57HS56 mini stepper motor: 1.2 N.m.

Since the motor torque is 1.2 N.m, which is greater than the torque requirement of 0.981 N.m, this motor is fully capable of rotating the mechanism.

Step 4: Calculate the power required for the engine

power P can be calculated from the torque and angular speed of the motor. The formula for calculating power is:

$$P = T \times \omega \quad P = T \times \omega$$

In there:

- $T = 0.981 \text{ N} \cdot \text{m}$ is the required torque,
- ω is the angular speed (rad/s), calculated from the engine RPM.

Assuming the motor operates at 1,200 RPM (RPM of the 57HS56 stepper motor), we have:

$$\omega = 2\pi \times 1200 / 60 = 125.66 \text{ rad/s}$$

Calculate required power:

$$P = 0.981 \times 125.66 \approx 123.6 \text{ W}$$

Conclude

- Torque required to rotate the mechanism: 0.981 N.m.
- The maximum torque of the 57HS56 mini stepper motor is 1.2 N.m, which far exceeds the requirement, so the motor can meet your requirements well.

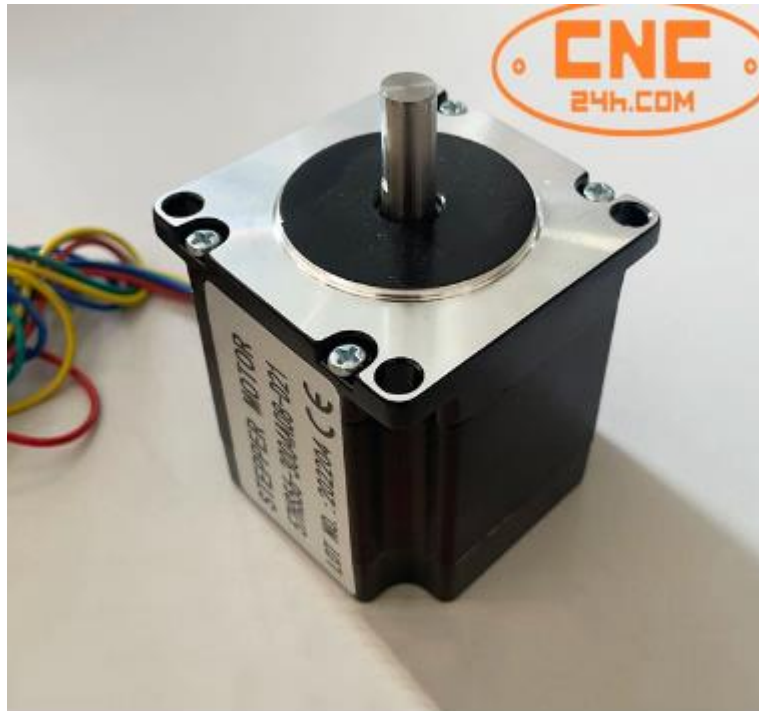


Figure 4.5 Step motor 57HS56

Mini Stepper Motor , 1.2Nm, 57HS56 features:

- Stepper motor has long life, runs smoothly, does not heat up
- Precise control of rotation angle, low noise
- Used in many mini CNC machines, laser engraving machines, small automation machines
- Is a 2-phase stepper motor, step angle 1.8 degrees
- Motor shaft D=6.35mm or D=8mm, keyway
- Motor size: Flange 57mmx57mm, body length 56mm
- Load capacity 3A, torque 1.2Nm, 4 wires, weight 680g

4.5 Firing direction rotation mechanism

Input data:

- Radius of rotating mechanism: 10 cm = 0.1 m
- Weight of rotating mechanism: 2 kg
- Acceleration of gravity: $g=9.81 \text{ m/s}^2$

Step 1: Calculate the force acting on the rotating mechanism

The acting force F can be calculated from the gravity formula:

$$F = m \times g$$

In there:

- $m = 2 \text{ kg}$ is the weight of the structure,
- $g = 9.81 \text{ m/s}^2$ is the acceleration of gravity.

Calculate the impact force:

$$F = 2 \times 9.81 = 19.62 \text{ N}$$

Step 2: Calculate the required torque

The torque T required to rotate the mechanism can be calculated from the formula:

$$T = F \times r$$

In there:

- $F = 19.62 \text{ N}$ $F = 19.62 \text{ N}$ is the force acting on the structure,
- $r = 0.1 \text{ m}$ $r = 0.1 \text{ m}$ is the radius of the rotating mechanism.

Calculate the required torque:

$$T = 19.62 \times 0.1 = 1.962 \text{ N} \cdot \text{m}$$

Step 3: Compare torque

- Torque required to rotate the mechanism: 1,962 N.m.
- Maximum torque of mini57HBP stepper motor: 1.8 N.m.

With a maximum motor torque of 2.2 N.m, the torque required to rotate the mechanism is 1,962 N.m, which is less than the maximum motor torque, which means that the motor The mechanism is capable of rotating a mechanism with a weight of 2 kg and a radius of 10 cm.

Step 4: Calculate the power required for the engine

power P can be calculated from the torque and angular speed of the motor. The formula for calculating power is:

$$P = T \times \omega \quad P = T \times \omega$$

In there:

- $T=1.962 \text{ N} \cdot \text{m}$ $T=1.962 \text{ N} \cdot \text{m}$ is the required torque,
- ω is the angular speed (rad/s), calculated from the engine RPM.

Assuming the motor operates at 1,200 RPM (typical RPM for the 57HS7630 stepper motor), we have:

$$\omega=2\pi \times 1200/60=125.66 \text{ rad/s}$$

Calculate required power:

$$P=1.962 \times 125.66 \approx 246.9 \text{ W}$$

Conclude

- Torque required to rotate the mechanism: 2.2 N.m.
- Maximum torque of the 57HBP stepper motor : 2.2 N.m, enough to rotate a mechanism with a weight of 2 kg and a radius of 10 cm.
- Power required: Approximately 246.9 W if using the motor at 1,200 RPM.



Figure 4.6 Step motor 57HBP

stepper motor, torque 2.2Nm 57HBP

- Dimensions: Flange 57mmx57mm, body length 76mm, shaft diameter 8mm
- Load capacity 3A, torque 1.8Nm, 4 wires, weight 1050g
- Genuine stepper motor, smooth running, stable, low noise
- Stepper motor runs without getting hot, long life
- Precise control of rotation angle
- Stepper motor size 57 is used in automation machines, laser engraving machines, small CNC machines...

CHAPTER 5: MECHANICAL DESIGN

Model after design completion

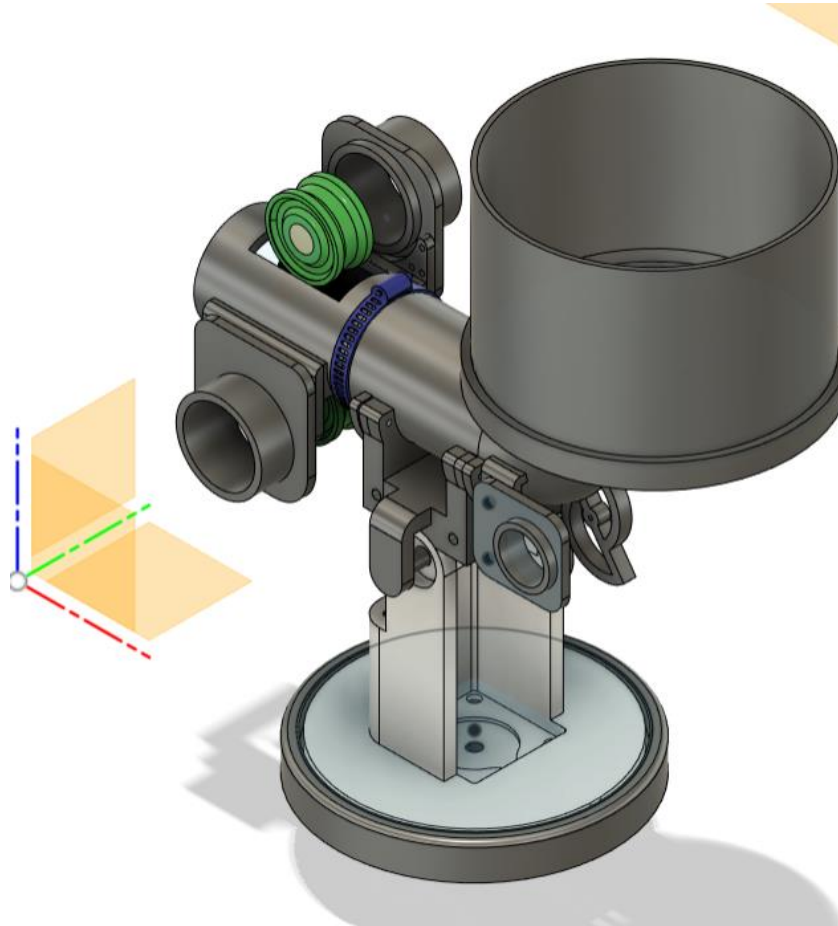


Figure 5.1 Complete model

5.1 Ball reversal mechanism



Figure 5.2 Ball leveling mechanism

Balls are contained in a basket with a DC gear motor. The ball-turning function sequentially brings the ball down to the ball feeder to avoid uneven ball output and ball blocking.

5.2. Ball pushing mechanism

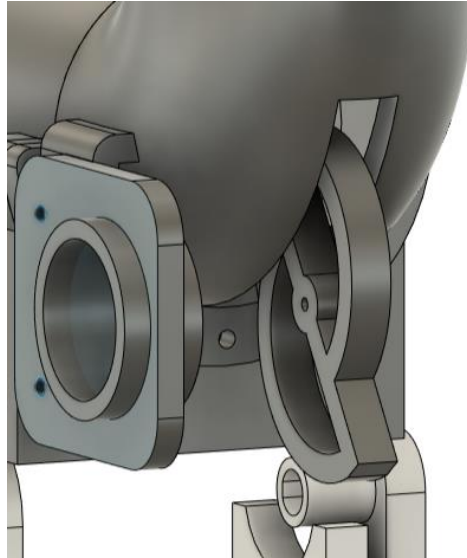


Figure 5.3 Ball push mechanism

Including a tube, a DC reducer to sequentially bring the ball to the barrel.

5.3 Ball shooting mechanism

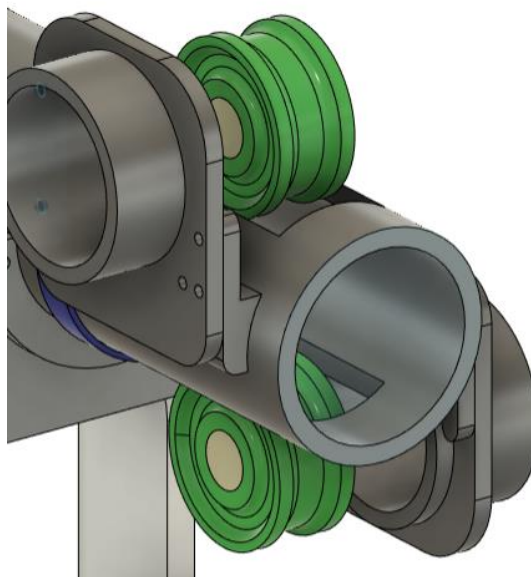


Figure 5.4 Ball shooting mechanism

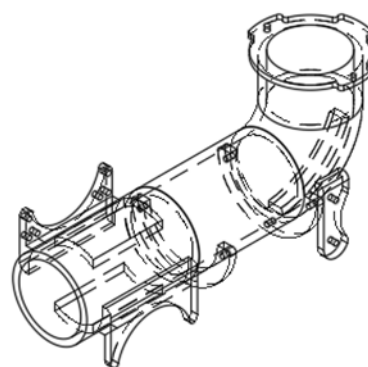
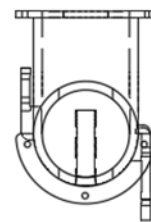
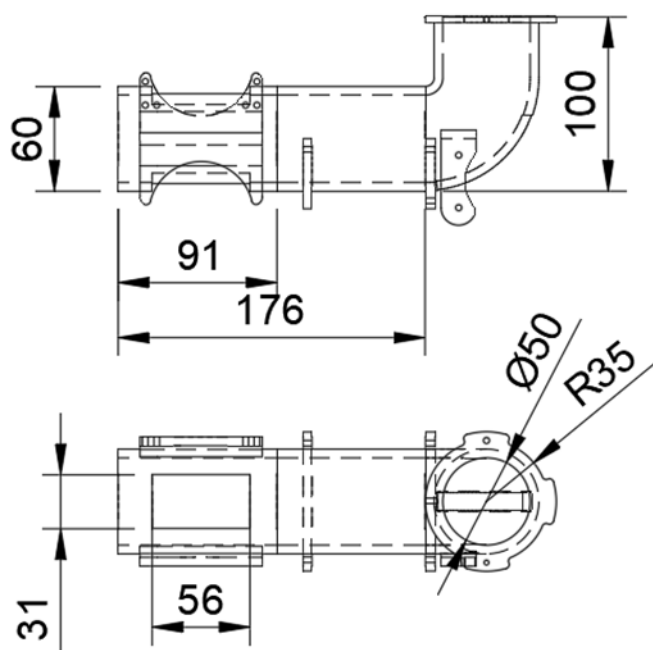
Includes 2 DC motors and 2 ball shooting wheels. The best material for making wheels is rubber. The shooting mechanism uses 2 motors to create friction like the racket face hitting the ball. The motor spins up and always rotates quickly to create friction force, pressing on the ball to push the ball out.

5.4 Machine operating principle

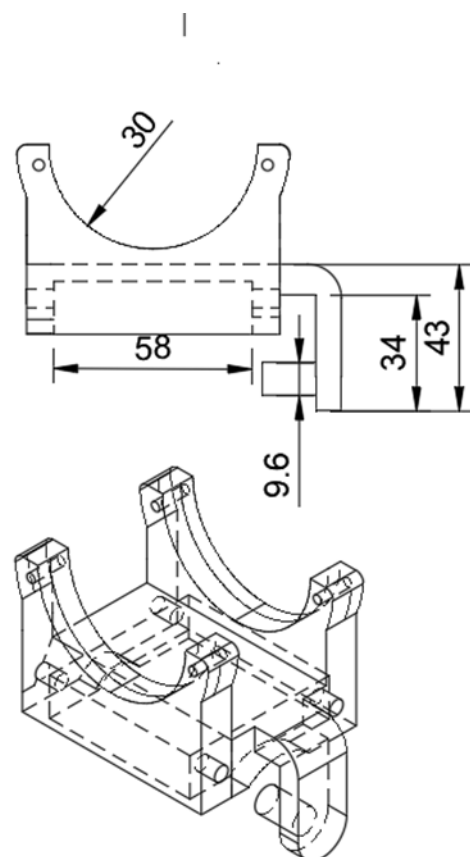
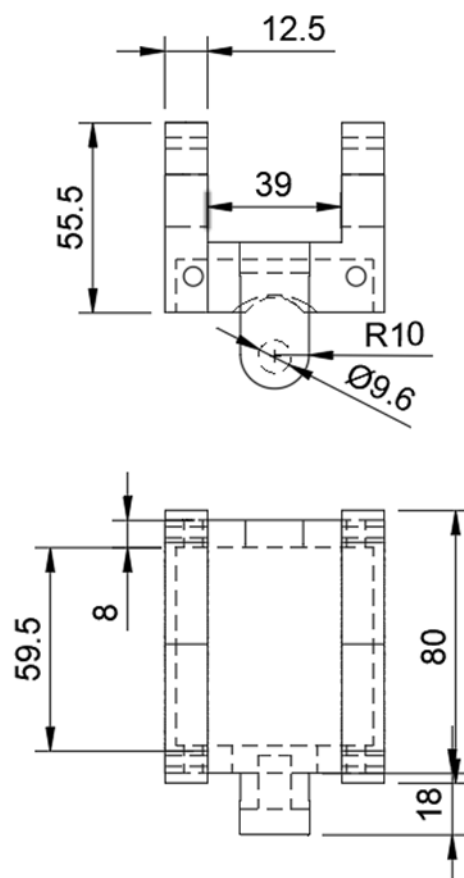
The ball is contained in the ball hopper, and will fall into the shooting tube thanks to the ball stirring motor. Then the ball will stop at the ball pushing position and thanks to the DC motor rotating to decide whether the ball is sent to the shooting part or not, the purpose of this position is to prevent the ball from falling down the shooting part, when the DC motor rotates the ball will be sent to the shooting part. The ball will be shot out thanks to the ball shooting mechanism with 2 ball shooting motors. The ball can swing left or right thanks to the shooting direction rotation mechanism as well as the shooting angle raising and lowering mechanism.

5.5 Technical drawing

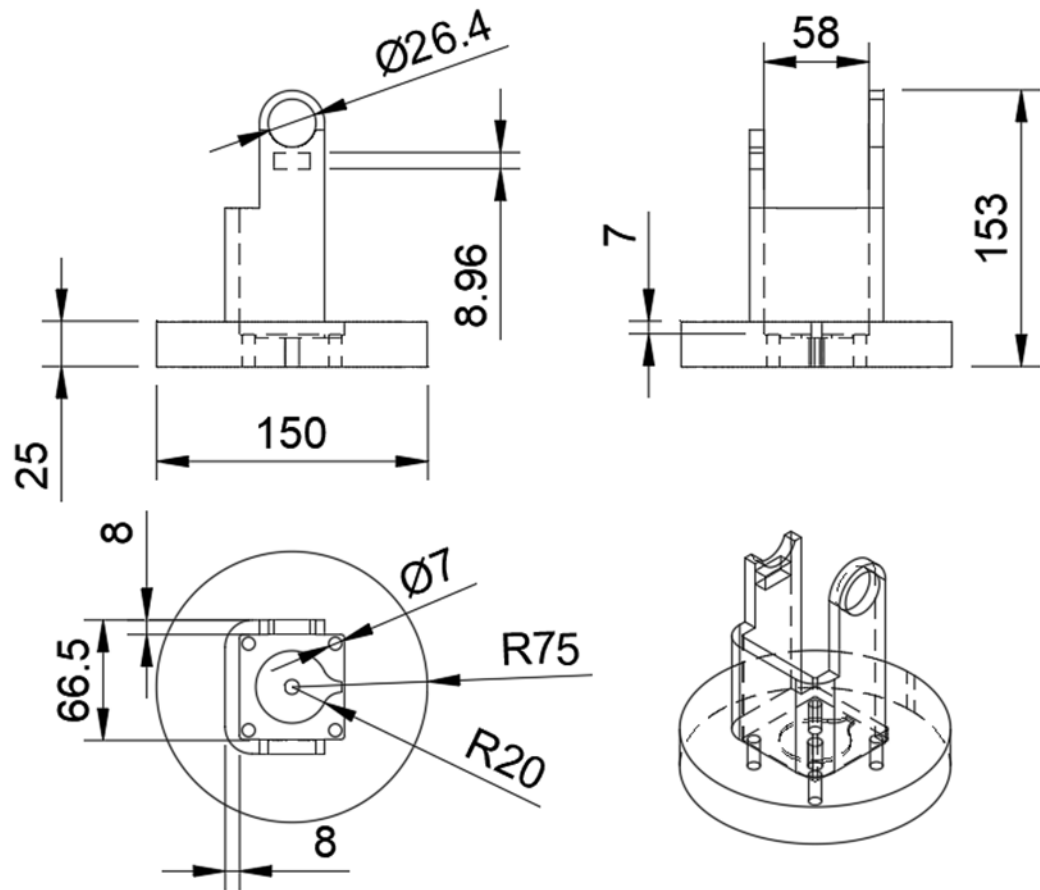
- Shooting barrel and shooting parts



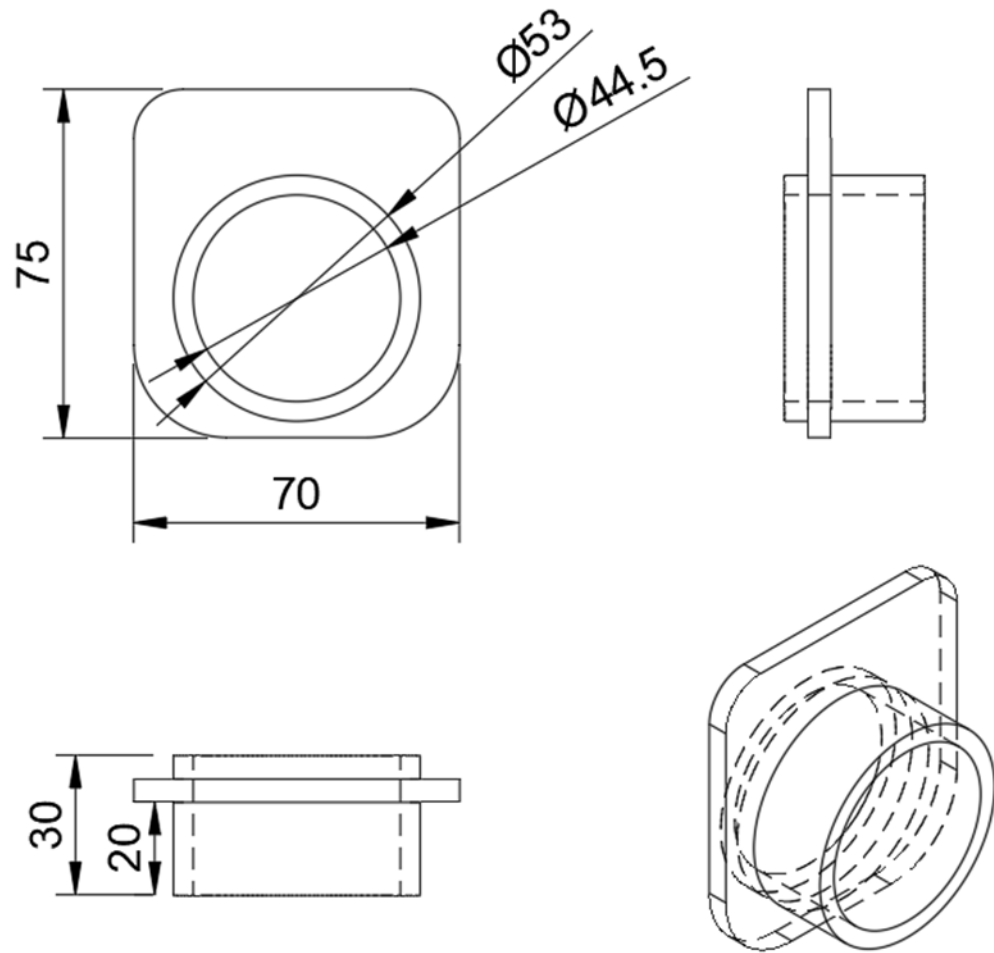
- angle lift mechanism



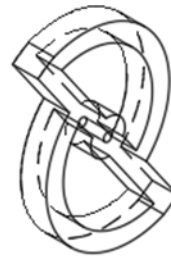
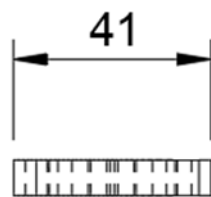
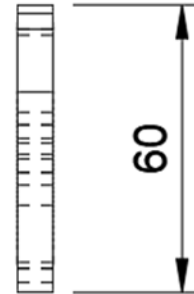
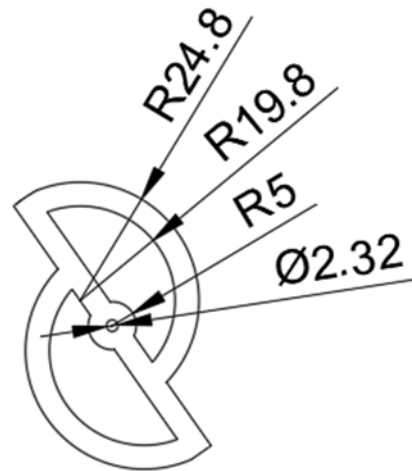
-
- rotation mechanism



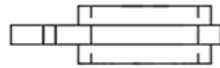
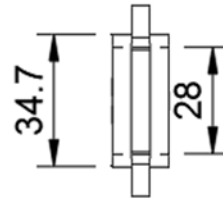
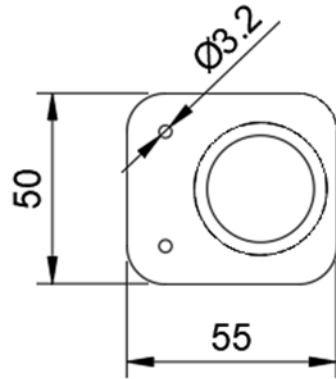
- DCRS775 motor mount



-
- ball push mechanism



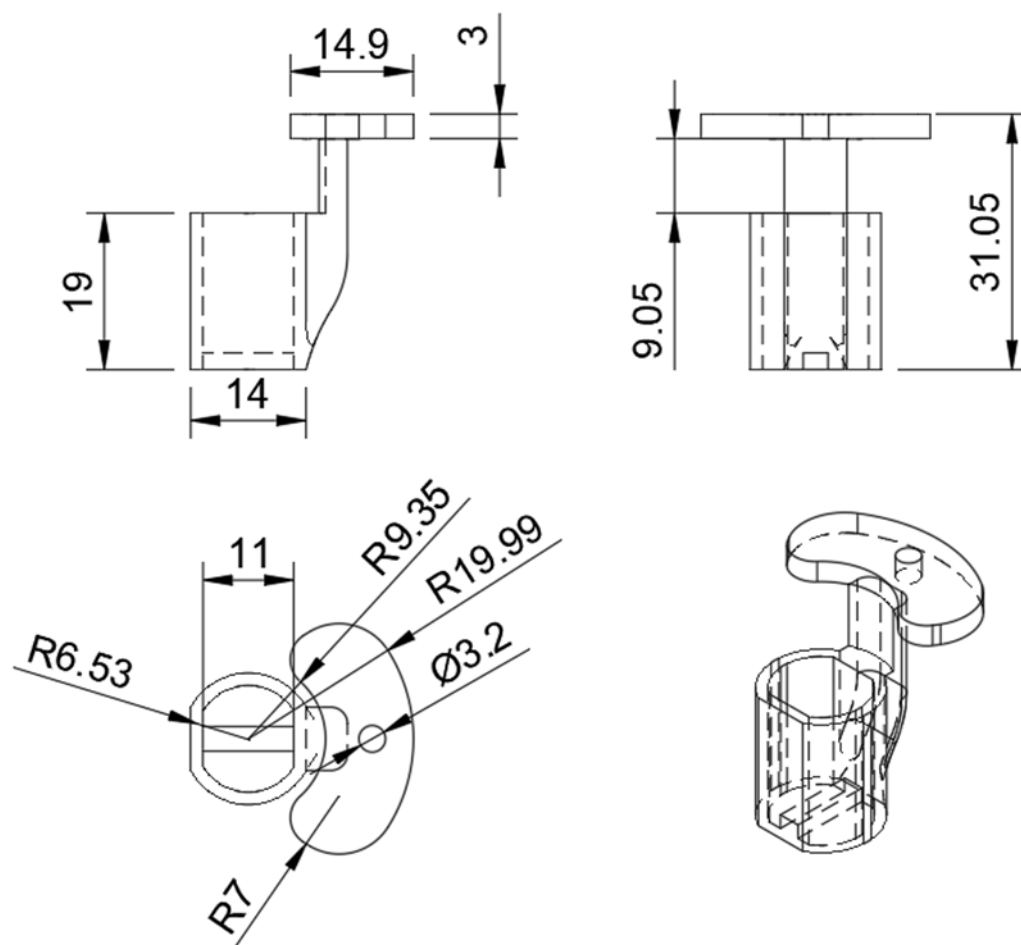
-
- DCRS335 motor mount

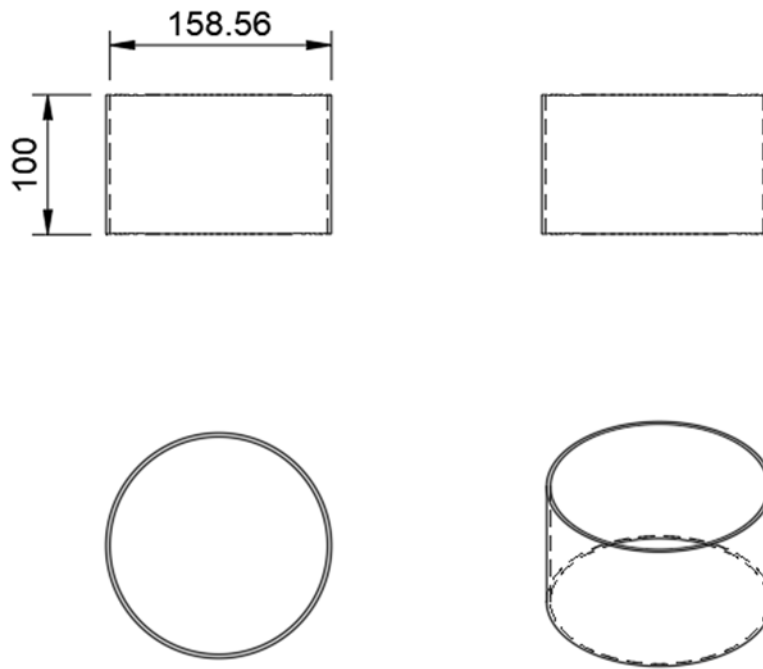


-

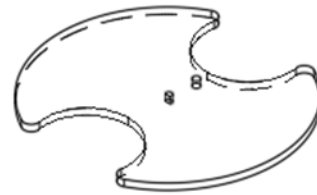
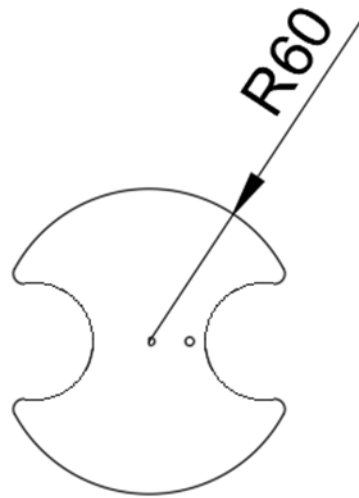
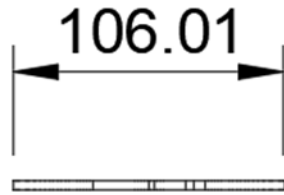
- ball basket

-





- GA12-N20 engine mount
- ball mechanism



5.6 Real-Life Model



Figure 5.5: Real-Life Model

CHAPTER 6: ELECTRICAL SYSTEM

6.1 Components Used in Electrical Systems

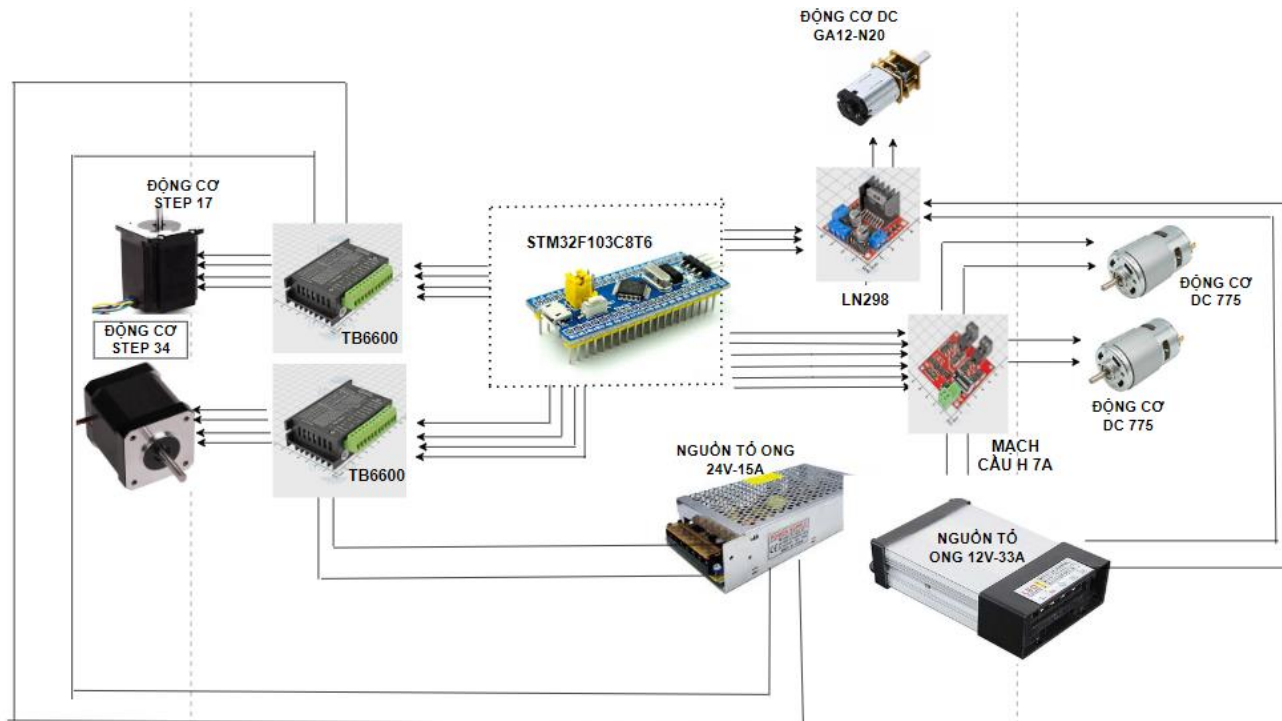


Figure 6.1: Connect System

6.1.1 DC Motor Driver Circuit - XY-160D

The XY-160D is a motor driver designed to control DC motors, providing an interface for controlling the direction and speed of the motor. It typically uses Pulse Width Modulation (PWM) to regulate the speed and direction of the motor. Below is a brief introduction and explanation of the components and features of the XY-160D DC Motor Driver:

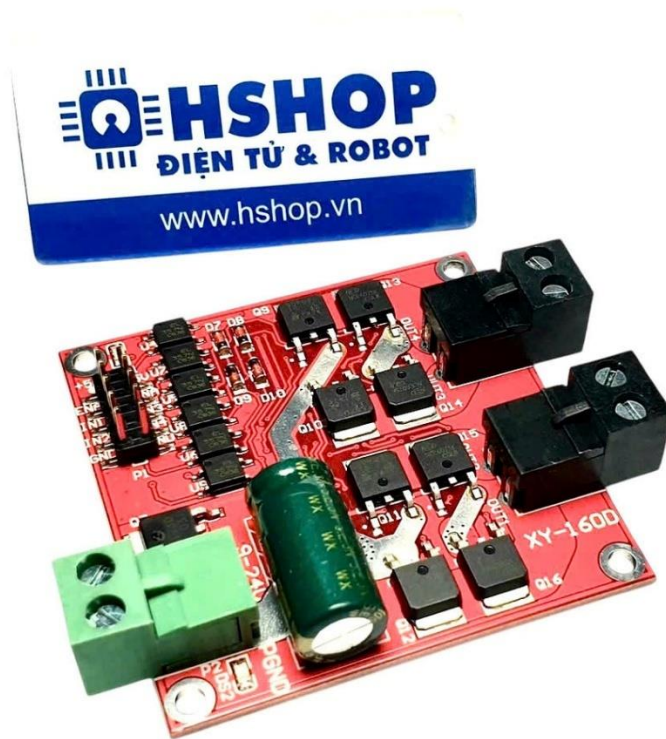


Figure 6.2: DC Motor Driver Circuit - XY-160D

Pin Configuration:

- **VCC:** Connect to the motor power supply (typically 6V to 40V DC).
- **GND:** Ground connection for the motor and control circuitry.
- **IN1 and IN2:** Control pins for motor direction (IN1 and IN2 work together to control the direction of the motor).
- **PWM:** Input pin for speed control via PWM signal (typically connected to a microcontroller or PWM generator).
- **Motor Outputs (M+) and (M-):** Motor terminals for connecting the DC motor.

How to Use:

1. **Power Supply:** Connect the VCC pin to the positive terminal of the DC power supply and the GND pin to the negative terminal.
2. **Motor Connection:** Connect the motor terminals (M+ and M-) to the DC motor.

3. **Control Pins:** Connect the IN1, IN2 pins to the GPIO pins of a microcontroller (e.g., Arduino) to control the direction. The PWM pin can be connected to a PWM-capable pin on the microcontroller to adjust the motor speed.
4. **Programming:** Program the microcontroller to send control signals for motor direction and speed, using PWM for speed control and digital signals for direction control.

6.1.2 L298 DC Motor Driver

- The **L298** is a dual H-bridge motor driver IC, widely used for driving DC motors, stepper motors, and other inductive loads. It allows for the control of motor direction and speed, making it an essential component in many robotics, automation, and DIY projects. Here's a brief overview of the **L298 DC Motor Driver** and its features:

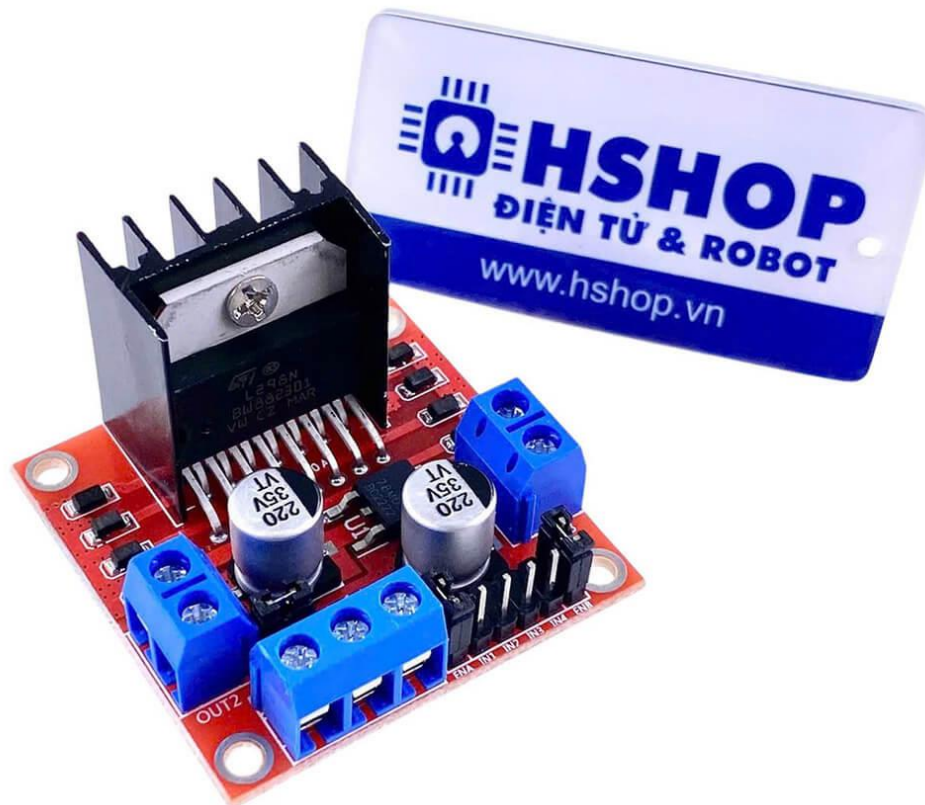


Figure 6.3: L298 DC Motor Driver

Pin Configuration:

- **Vcc (Pin 4, 5):** Motor power supply (typically between 5V and 46V).
- **GND (Pin 8):** Ground connection for the driver.

- **IN1 (Pin 2) and IN2 (Pin 7):** Input pins for controlling the direction of Motor A.
- **IN3 (Pin 10) and IN4 (Pin 15):** Input pins for controlling the direction of Motor B.
- **ENA (Pin 1) and ENB (Pin 9):** Enable pins for controlling motor operation (typically connected to Vcc for enabling).
- **OUT1 (Pin 3) and OUT2 (Pin 6):** Motor A output terminals.
- **OUT3 (Pin 11) and OUT4 (Pin 14):** Motor B output terminals.
- **Vss (Pin 5):** Logic voltage supply (usually 5V).

How to Use:

1. **Connect Power Supply:**

Connect the **Vcc** pin to the motor power supply (6V to 46V), and the **GND** pin to the ground of the power source. For the logic supply, connect **Vss** to 5V.

2. **Motor Connection:**

Connect the motor terminals to the output pins **OUT1**, **OUT2** (for Motor A), and **OUT3**, **OUT4** (for Motor B).

3. **Control Pins:**

- Connect the **IN1**, **IN2** pins to the GPIO pins of a microcontroller (such as Arduino) to control the direction of Motor A.
- Connect the **IN3**, **IN4** pins to the GPIO pins of the microcontroller to control the direction of Motor B.
- Use PWM signals for the **ENA** and **ENB** pins to control the speed of the motors.

4. **Programming:**

Program the microcontroller to send HIGH/LOW signals for direction control and PWM signals for speed control.

6.1.3 Mini PWM DC Motor Speed Controller 10A Circuit

The **Mini PWM DC Motor Speed Controller 10A** is a versatile and compact circuit designed for controlling the speed of DC motors using Pulse Width Modulation (PWM) technique. It is ideal for small to medium DC motors, and its key feature is its ability to adjust motor speed efficiently with minimal heat generation.



Figure 6.4: Mini PWM DC Motor Speed Controller

Mini PWM DC Motor Speed Controller 10A Circuit

The **Mini PWM DC Motor Speed Controller 10A** is a versatile and compact circuit designed for controlling the speed of DC motors using Pulse Width Modulation (PWM) technique. It is ideal for small to medium DC motors, and its key feature is its ability to adjust motor speed efficiently with minimal heat generation.

6.1.4 Stepper Motor Driver Microstep 3.5A 40VDC TB6600

The **TB6600 Stepper Motor Driver** is a popular and powerful driver used for controlling stepper motors with a wide range of features, including **microstepping** control, efficient power handling, and adjustable current settings. This driver is suitable for high-performance stepper motors in various applications, including robotics, CNC machines, 3D printers, and other automation projects.



Figure 6.5: Driver TB6600

Pin Configuration:

The TB6600 Stepper Motor Driver has several key pins for control and connection:

- **VCC:** Power supply input (DC voltage 9V to 40V).
- **GND:** Ground pin.
- **A+, A-:** Motor phase A connections.
- **B+, B-:** Motor phase B connections.
- **STEP:** Pulse input pin (controls the movement of the motor).
- **DIR:** Direction input pin (sets the motor's rotation direction).
- **EN:** Enable input pin (enables or disables the motor driver).
- **NC:** Not connected (often used for ground).
- **Current Adjustment:** A potentiometer is used to set the current limit for the motor.

Working Principle:

1. Pulse and Direction Control:

The TB6600 stepper motor driver operates by receiving **step** and **direction** signals from a microcontroller. The **STEP** signal tells the driver when to advance the motor one step, while the **DIR** signal determines the direction of rotation. The microcontroller sends a series of pulses to the step input, and each pulse moves the motor one step forward (or backward, depending on the direction signal).

2. Microstepping:

By using **microstepping**, the driver divides each full step of the motor into smaller sub-steps, resulting in smoother and more precise motion. The TB6600 can work in several microstepping modes (e.g., full, half, quarter, eighth, or sixteenth step), allowing the user to choose the level of precision and smoothness required for the application.

3. Current Limiting:

The current limit can be adjusted by the user to match the specifications of the motor. The potentiometer on the driver allows the user to fine-tune the current delivered to the motor to avoid overheating and ensure optimal performance.

4. Safety Features:

The TB6600 includes protection mechanisms to prevent damage due to overcurrent, overvoltage, and overheating. If any of these conditions are detected, the driver will automatically shut down or limit power to protect both the motor and the driver.

CHAPTER 7: APPLYING MATLAB TO SIMULATE TRAVELS, CALCULATE RELATED PARAMETERS

7.1 Simulation for Obtaining Data for Motor Control Coding

When using simulation to collect data for programming motor control, the main goal is to determine the input and output parameters related to the motors, such as **launch angles**, **motor speed (RPM)**, and **spin direction**. Below is a guide on how you can report or organize this simulation:

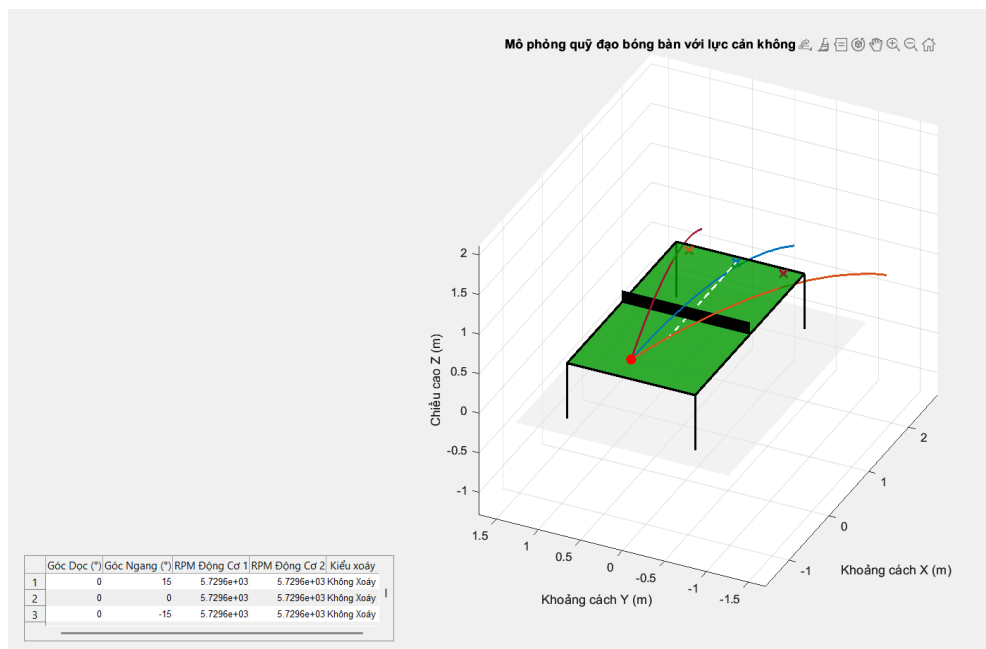


Figure 7.1: Example Results from Simulation

CODE MATLAB:

% Mô phỏng quỹ đạo bóng bàn với lực cản không khí và lực Magnus

clear; close all; clc;

%% Thông số đầu vào

% Vật lý

m = 0.0027; % Khối lượng quả bóng (kg)

D = 0.04; % Đường kính quả bóng (m)

Cd = 0.47; % Hệ số cản không khí

rho = 1.225; % Mật độ không khí (kg/m³)

g = 9.81; % Gia tốc trọng trường (m/s²)

% Bắn bóng

v0 = 12; % Vận tốc ban đầu (m/s)

angles = [0,0,0]; % Các góc bắn theo phương dọc (độ)

phi_angles = [15,0,-15]; % Các góc bắn theo phương ngang (độ) (dương trái , thẳng, âm phải)

% Bánh xe và xoáy

Dp = 0.04; % Đường kính bánh xe bắn bóng (m)

spin_type = 'none'; % 'none', 'Trai', 'Phai', 'Len', 'Xuong'

spin = 1200; % Số vòng xoay mỗi phút (RPM) - cường độ xoáy

% Chuyển đổi độ xoáy sang tốc độ góc

omega_mag = spin * 2 * pi / 60; % Tốc độ góc (rad/s)

% Tính diện tích mặt cắt ngang của quả bóng

A = pi * (D/2)²;

% Hằng số Magnus (cần hiệu chỉnh dựa trên thực nghiệm)

Cm = 5.15e-5; % Giá trị ước lượng ban đầu

% Hệ số phục hồi sau va chạm ($0 \leq e \leq 1$)

e = 0.9; % 90% vận tốc giữ lại sau va chạm

%% Thông số bàn bóng bàn

table_length = 2.74; % m

table_width = 1.525; % m

table_height = 0.76; % m

net_height = 0.1525; % m

leg_height = 0.7; % Chiều cao chân bàn (m)

%% Xác định các điểm mục tiêu trên bàn đối phương

% Chia bàn đối phương thành một lưới 3x3

num_targets_x = 3;

num_targets_y = 3;

target_distance = -0.1; % Khoảng cách từ bàn đối phương (m)

target_x = table_length/2 + target_distance * ones(1, num_targets_x);

target_y = linspace(-table_width/2 + 0.2, table_width/2 - 0.2, num_targets_y);

[target_grid_x, target_grid_y] = meshgrid(target_x, target_y);

targets = [target_grid_x(:), target_grid_y(:)];

%% Số lượng quỹ đạo sẽ mô phỏng

num_trajectories = size(targets, 1);


```
%% Thiết lập đồ họa
figure;
hold on;
grid on;
axis equal;
xlabel('Khoảng cách X (m)');
ylabel('Khoảng cách Y (m)');
zlabel('Chiều cao Z (m)');
title('Mô phỏng quỹ đạo bóng bàn với lực cản không khí và lực Magnus');
view(01, 01);

%% Vẽ bàn bóng bàn và lưới
draw_table(table_length, table_width, table_height, net_height, leg_height);

%% Vẽ các quỹ đạo mục tiêu
colors = lines(num_trajectories); % Màu sắc cho từng quỹ đạo
legend_labels = {};

% Vòng lặp qua từng điểm mục tiêu
for i = 1:num_trajectories
    target = targets(i, :);
    theta = angles(mod(i-1, length(angles)) + 1); % Góc bắn theo phương dọc
```

```
phi = phi_angles(mod(i-1, length(phi_angles)) + 1); % Góc bắn theo phương ngang
```

```
% Tính vận tốc ban đầu theo từng trục
```

```
vx0 = v0 * cosd(theta) * cosd(phi);
```

```
vy0 = v0 * cosd(theta) * sind(phi);
```

```
vz0 = v0 * sind(theta);
```

```
% Tính toán tốc độ của hai động cơ dựa trên spin_type và spin
```

```
base_RPM = (v0 * 60) / (pi * Dp); % RPM cơ bản dựa trên vận tốc ban đầu
```

```
switch spin_type
```

```
case 'Xuong'
```

```
    motor1_RPM = base_RPM + spin/2; % Tăng RPM động cơ 1
```

```
    motor2_RPM = base_RPM - spin/2; % Giảm RPM động cơ 2
```

```
    omega_vec = [0; omega_mag; 0]; % Xoay trái
```

```
case 'Len'
```

```
    motor1_RPM = base_RPM - spin/2; % Giảm RPM động cơ 1
```

```
    motor2_RPM = base_RPM + spin/2; % Tăng RPM động cơ 2
```

```
    omega_vec = [0; -omega_mag; 0]; % Xoay phải
```

```
case 'Trai'
```

```
    motor1_RPM = base_RPM + spin/2; % Tăng tốc độ bánh xe 1
```

```
    motor2_RPM = base_RPM - spin/2; % Giảm tốc độ bánh xe 2
```

```
    omega_vec = [omega_mag; 0; 0]; % Xoay quanh trục X
```

```
case 'Phai'
```

```
motor1_RPM = base_RPM - spin/2; % Giảm tốc độ bánh xe 1
motor2_RPM = base_RPM + spin/2; % Tăng tốc độ bánh xe 2
omega_vec = [-omega_mag; 0; 0]; % Xoay quanh trục X ngược chiều
otherwise
    motor1_RPM = base_RPM;
    motor2_RPM = base_RPM;
    omega_vec = [0; 0; 0]; % Không xoay
end

% Điều kiện ban đầu [x; y; z; vx; vy; vz]
initial_conditions = [-table_length/2; 0; table_height + 0.25; vx0; vy0; vz0]; % Nâng
cao z lên 20cm

% Mô phỏng toàn bộ quỹ đạo với va chạm
[x_total, y_total, z_total] = simulate_trajectory(initial_conditions, m, rho, Cd, A,
omega_vec, Cm, g, e);

% Vẽ quỹ đạo
plot3(x_total, y_total, z_total, 'Color', colors(i,:), 'LineWidth', 1.5);

% Vẽ điểm mục tiêu
plot3(target(1), target(2), table_height, 'x', 'Color', colors(i,:), 'MarkerSize', 10,
'LineWidth', 2);

% Vẽ điểm bắt đầu (chỉ vẽ một lần)
```

```
    if i == 1
        plot3(x_total(1), y_total(1), z_total(1), 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
    end

    % Ghi nhãn cho legend
    legend_labels{end+1} = sprintf('Góc %.0f°, Phi %.0f°', theta, phi);
end

% Thêm legend
legend(legend_labels, 'Location', 'best');

%% Thêm bảng hiển thị góc bắn, phi, vận tốc và số vòng quay của hai động cơ
% Chuẩn bị dữ liệu cho bảng
data_table = [];
for i = 1:num_trajectories
    theta = angles(mod(i-1, length(angles)) + 1);
    phi = phi_angles(mod(i-1, length(phi_angles)) + 1);
    data_table = [data_table; theta, phi, v0, motor1_RPM, motor2_RPM];
end

% Loại bỏ các bản ghi trùng lặp nếu có
[data_table_unique, ~, ~] = unique(data_table, 'rows');

% Thêm bảng hiển thị
```

```
%uitable('Data', data_table_unique, ...  
    % 'ColumnName', {'Góc bắn (°)', 'Phi (°)', 'Vận tốc (m/s)', 'RPM động cơ 1', 'RPM  
động cơ 2'}, ...  
    % 'Position', [20, 20, 600, 200]);  
  
%hold off;  
  
% Khởi tạo bảng dữ liệu tổng hợp  
data_table = [];  
  
% Lặp qua từng quỹ đạo và thu thập dữ liệu  
for i = 1:num_trajectories  
    target = targets(i, :); % Lấy tọa độ của điểm mục tiêu  
    theta = angles(mod(i-1, length(angles)) + 1); % Góc bắn theo phương dọc  
    phi = phi_angles(mod(i-1, length(phi_angles)) + 1); % Góc bắn theo phương ngang  
  
    % Gán tốc độ của hai động cơ dựa trên hướng xoay  
    switch spin_type  
        case 'Trai'  
            motor1_RPM = base_RPM + spin/2;  
            motor2_RPM = base_RPM - spin/2;  
            direction = 'Trái';  
        case 'Phai'  
            motor1_RPM = base_RPM - spin/2;  
            motor2_RPM = base_RPM + spin/2;  
            direction = 'Phải';
```

```
case 'top'
    motor1_RPM = base_RPM + spin/2;
    motor2_RPM = base_RPM - spin/2;
    direction = ' Xoáy Trên';
case 'back'
    motor1_RPM = base_RPM - spin/2;
    motor2_RPM = base_RPM + spin/2;
    direction = ' Xoáy Ngược';
otherwise
    motor1_RPM = base_RPM;
    motor2_RPM = base_RPM;
    direction = 'Không Xoáy';
end

% Thêm dữ liệu vào bảng
data_table = [data_table; theta, phi, motor1_RPM, motor2_RPM, {direction}];

end

% Tạo bảng hiển thị trong MATLAB
uitable('Data', data_table, ...
    'ColumnName', { 'Góc Dọc (°)', 'Góc Ngang (°)', 'RPM Động Cơ 1', 'RPM Động Cơ 2', 'Kiểu xoáy'}, ...
    'Position', [20, 20, 420, 98]);
```

%% Định Nghĩa Hàm

% Hàm mô phỏng toàn bộ quỹ đạo bao gồm va chạm

```
function [x_total, y_total, z_total] = simulate_trajectory(initial_conditions, m, rho, Cd, A,  
omega_vec, Cm, g, e)
```

% Khởi tạo các vector để lưu trữ kết quả

```
x_total = [];
```

```
y_total = [];
```

```
z_total = [];
```

% Thiết lập trạng thái ban đầu

```
current_state = initial_conditions;
```

% Thiết lập thời gian mô phỏng tối đa

```
t_max_total = 10; % Giờ xem xét tối đa 10 giây
```

```
t_current = 0;
```

```
while t_current < t_max_total
```

% Thời gian giải tích cho giai đoạn tiếp theo

% Giới hạn thời gian giải tiếp theo để tránh chạy vô hạn

```
t_span = [t_current, t_max_total];
```

% Sử dụng ode45 để giải phương trình chuyển động

```
options = odeset('Events', @(t,S) eventFcn(t,S));
```

```
[t_vals, S] = ode45(@(t,S) motion(t, S, m, rho, Cd, A, omega_vec, Cm, g), t_span,  
current_state, options);
```

```
% Trích xuất vị trí
```

```
x_vals = S(:,1);
```

```
y_vals = S(:,2);
```

```
z_vals = S(:,3);
```

```
% Thêm kết quả vào tổng quỹ đạo
```

```
x_total = [x_total; x_vals];
```

```
y_total = [y_total; y_vals];
```

```
z_total = [z_total; z_vals];
```

```
% Cập nhật thời gian hiện tại
```

```
t_current = t_vals(end);
```

```
% Kiểm tra xem có xảy ra va chạm không
```

```
if length(t_vals) < 2
```

```
    % Không có va chạm, kết thúc mô phỏng
```

```
    break;
```

```
end
```

```
if S(end,3) <= 0
```

```
    % Va chạm đã xảy ra
```

```
    % Cập nhật trạng thái sau va chạm
```



```
current_state = [S(end,1); S(end,2); 0; S(end,4); S(end,5); -e * S(end,6)];

% Kiểm tra nếu vận tốc sau va chạm quá nhỏ, kết thúc mô phỏng
if abs(current_state(6)) < 0.1
    break;
end
else
    % Không có va chạm, kết thúc mô phỏng
    break;
end
end
end

% Hàm mô hình hóa chuyển động
function dSdt = motion(~, S, m, rho, Cd, A, omega_vec, Cm, g)
    % S = [x; y; z; vx; vy; vz]
    vx = S(4);
    vy = S(5);
    vz = S(6);

    v = sqrt(vx^2 + vy^2 + vz^2);
    if v == 0
        v = 1e-8; % Tránh chia cho 0
    end
```

% Lực cản không khí

$$F_d = 0.5 * \rho * C_d * A * v^2;$$

$$F_{dx} = -F_d * (v_x / v);$$

$$F_{dy} = -F_d * (v_y / v);$$

$$F_{dz} = -F_d * (v_z / v);$$

% Lực Magnus

$$F_m = C_m * \text{cross}(\omega_vec, [v_x; v_y; v_z]);$$

$$F_{mx} = F_m(1);$$

$$F_{my} = F_m(2);$$

$$F_{mz} = F_m(3);$$

% Tổng lực

$$a_x = (F_{dx} + F_{mx}) / m;$$

$$a_y = (F_{dy} + F_{my}) / m;$$

$$a_z = (F_{dz} + F_{mz}) / m - g;$$

% Trả về đạo hàm

$$dSdt = [v_x; v_y; v_z; a_x; a_y; a_z];$$

end

% Hàm dừng khi quả bóng chạm đất

```
function [value, isterminal, direction] = eventFcn(~, S)
    value = S(3);    % Chiều cao z
    isterminal = 1;  % Dừng khi xảy ra sự kiện
    direction = -1;  % Chỉ dừng khi z giảm xuống
end
```

% Hàm vẽ bàn bóng bàn với lưới chi tiết

```
function draw_table(table_length, table_width, table_height, net_height, leg_height)
    % Vẽ mặt bàn
    x = [-table_length/2, table_length/2, table_length/2, -table_length/2, -table_length/2];
    y = [-table_width/2, -table_width/2, table_width/2, table_width/2, -table_width/2];
    z = [table_height, table_height, table_height, table_height, table_height];
    fill3(x, y, z, [0.0 0.6 0.0], 'FaceAlpha', 0.8, 'EdgeColor', 'k', 'LineWidth', 2); % Mặt bàn
    màu xanh lá
```

```
hold on;
```

% Vẽ các đường kẻ dịch vụ

```
num_service_lines = 2; % Số đường kẻ dịch vụ mỗi bên
service_spacing = table_length / (num_service_lines + 1);
for i = 1:num_service_lines
    service_x_start = -table_length/2 + i*service_spacing;
    service_x_end = table_length/2;
    service_y = 0;
    service_z = table_height;
```

```
    plot3([service_x_start, service_x_end], [service_y, service_y], [service_z,  
service_z], 'w--', 'LineWidth', 1.5); % Đường kẻ dịch vụ màu trắng  
end
```

```
% Vẽ lưới (net) dạng grid hình chữ nhật tại trung tâm bàn
```

```
% Định nghĩa số lượng đường ngang và dọc
```

```
num_net_h = 25; % Số đường ngang (theo chiều cao của lưới)
```

```
num_net_v = 25; % Số đường dọc (theo chiều rộng của lưới)
```

```
% Định nghĩa phạm vi lưới (tại x = 0, trung tâm bàn)
```

```
net_x = 0; % Vị trí trung tâm bàn
```

```
net_y_start = -table_width/2;
```

```
net_y_end = table_width/2;
```

```
net_z_start = table_height;
```

```
net_z_end = table_height + net_height;
```

```
% Định nghĩa khoảng cách giữa các đường kẻ
```

```
dy = (net_y_end - net_y_start) / (num_net_v + 1);
```

```
dz = (net_z_end - net_z_start) / (num_net_h + 1);
```

```
% Vẽ các đường dọc (theo y) trên lưới
```

```
for i = 1:num_net_v
```

```
    y = net_y_start + i*dy;
```

```
    plot3([net_x, net_x], [y, y], [net_z_start, net_z_end], 'k-', 'LineWidth', 1); % Đường  
dọc màu trắng
```

```
end

% Vẽ các đường ngang (theo z) trên lưới
for i = 1:num_net_h
    z = net_z_start + i*dz;
    plot3([net_x, net_x], [net_y_start, net_y_end], [z, z], 'k-', 'LineWidth', 1); % Đường
    ngang màu trắng
end

% Vẽ các cạnh của bàn (đường viền)

% Front edge
plot3([-table_length/2, table_length/2], [-table_width/2, -table_width/2], [table_height,
table_height], 'k', 'LineWidth', 2);

% Back edge
plot3([-table_length/2, table_length/2], [table_width/2, table_width/2], [table_height,
table_height], 'k', 'LineWidth', 2);

% Left edge
plot3([-table_length/2, -table_length/2], [-table_width/2, table_width/2], [table_height,
table_height], 'k', 'LineWidth', 2);

% Right edge
plot3([table_length/2, table_length/2], [-table_width/2, table_width/2], [table_height,
table_height], 'k', 'LineWidth', 2);

% Vẽ chân bàn (giả sử bàn có 4 chân, chân ở góc)
leg_positions = [
    -table_length/2, -table_width/2;
```

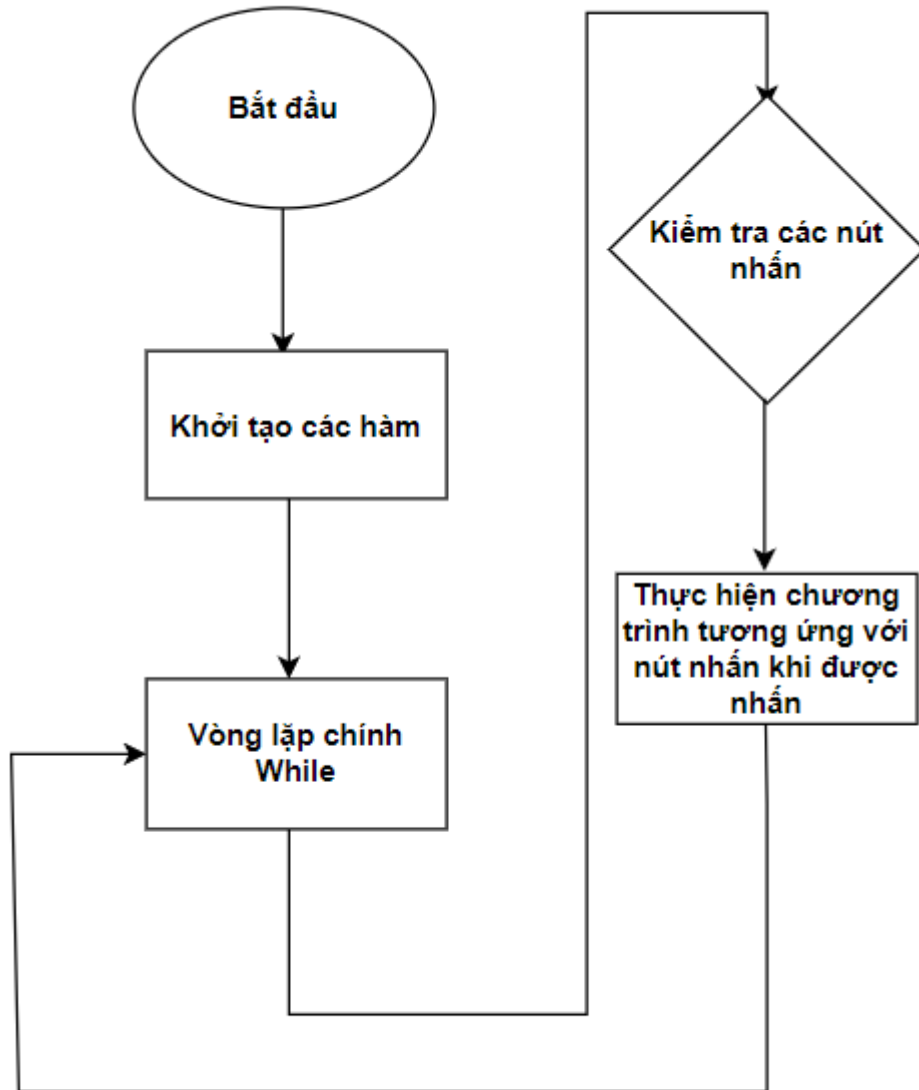
```
-table_length/2, table_width/2;  
table_length/2, -table_width/2;  
table_length/2, table_width/2  
];
```

```
for i = 1:size(leg_positions, 1)  
    x_leg = [leg_positions(i,1), leg_positions(i,1)];  
    y_leg = [leg_positions(i,2), leg_positions(i,2)];  
    z_leg = [table_height, table_height - leg_height];  
    plot3(x_leg, y_leg, z_leg, 'k', 'LineWidth', 2);  
end
```

```
% Vẽ mặt sân (tùy chọn, nếu bạn muốn có mặt sân)  
floor_length = table_length + 1; % Mở rộng mặt sân  
floor_width = table_width + 1;  
floor_x = [-floor_length/2, floor_length/2, floor_length/2, -floor_length/2, -  
floor_length/2];  
floor_y = [-floor_width/2, -floor_width/2, floor_width/2, floor_width/2, -  
floor_width/2];  
floor_z = [0, 0, 0, 0, 0];  
fill3(floor_x, floor_y, floor_z, [0.9 0.9 0.9], 'FaceAlpha', 0.5, 'EdgeColor', 'none'); %  
Mặt sân màu xám nhạt  
end
```


CHAPTER 8: PROGRAMMING AND CONTROL

Algorithm Flowchart:



- **STM32F103C8T6 Blue Pill** microcontroller chip for programming and control.



Figure 8.1 STM32F103C8T6 Microcontroller

```

- /* USER CODE BEGIN Header */
- /**
- /* USER CODE END Header */
- /* Includes -----*/
- #include "main.h"
-
- /* Private includes -----*/
- /* USER CODE BEGIN Includes */
- #include "step_motor.h" // Thư viện điều khiển động cơ bước
- /* USER CODE END Includes */
-
- /* Private typedef -----*/
- /* USER CODE BEGIN PTD */
-
- // Định nghĩa enum cho trạng thái của động cơ bước
- typedef enum {
-     STATE_STOP = 0,           // động cơ dừng
-     STATE_EXECUTE_STEP_SEQUENCE // động cơ đang thực hiện chuỗi bước
- } MotorState;
-
- // Cấu trúc lưu trữ thông tin với mẫu chuyển động của động cơ
- typedef struct {
-     uint8_t mode;           // Chế độ hiện tại (1, 2, 3, 4, 5)
-     uint8_t stepIndex;      // Chỉ số bước hiện tại trong mẫu (0-4)

```

```

-   uint32_t stepAngles[5]; // Mảng chứa góc cho từng bước trong mẫu
-   uint8_t stepDirections[5]; // Mảng chứa hướng cho từng bước trong mẫu (1: thuận, 0:
ngược)
-   uint8_t numSteps; // Số bước trong mẫu
-   } MotorPattern;
-
-   /* USER CODE END PTD */
-
-   /* Private define -----*/
-   /* USER CODE BEGIN PD */
-
-   #define STEPMOTOR_STEPS 200 // Số bước cho mỗi vòng quay của động cơ bước
-   #define MAX_SPEED_PERCENT 100 // Tốc độ tối đa (100%)
-
-   /* USER CODE END PD */
-
-   /* Private macro -----*/
-   /* USER CODE BEGIN PM */
-   /* USER CODE END PM */
-
-   /* Private variables -----*/
-   TIM_HandleTypeDef htim2;
-   TIM_HandleTypeDef htim3;
-
-   /* USER CODE BEGIN PV */
-
-   /* Biến toàn cục cho động cơ DC */
-   volatile uint16_t DCmode = 0; // Chế độ cho động cơ DC1 và DC2
-   volatile uint16_t DCmode1 = 0; // Chế độ cho động cơ DC3
-   volatile uint8_t t = 0; // C? để cập nhật chế độ cho DC1 và DC2
-   volatile uint8_t t1 = 0; // C? để cập nhật chế độ cho DC3
-
-   uint8_t CCRvalue = 0; // Giá trị so sánh cho PWM (không sử dụng trong code này)
-
-   // Khai báo đối tượng StepMotor cho động cơ bước thứ nhất (step1)
-   StepMotor step1 = {
-       .dirPort = GPIOB,
-       .dirPin = GPIO_PIN_12,
-       .enablePort = GPIOB,
-       .enablePin = GPIO_PIN_13,
-       .ratio = 1,
-       .htim = NULL, // Sẽ khởi tạo sau
-       .channel = TIM_CHANNEL_1,
-       .current_mode = MODE_STOP,

```

```
- .velocity = 0,
- .current_pulses = 0,
- .cnt = 0,
- .nbPulse = 0,
- .accPulse = 0,
- .decPulse = 0,
- .accSize = 0,
- .decSize = 0,
- .run = 0,
- .move = 0,
- .acc = 0,
- .stop = 0,
- .dec = 0,
- .softStop = 0,
- .arr = 0
- };
-
- // Khai báo đối tượng StepMotor cho động cơ bước thứ hai (step2)
- StepMotor step2 = {
- .dirPort = GPIOB,
- .dirPin = GPIO_PIN_14,
- .enablePort = GPIOB,
- .enablePin = GPIO_PIN_15,
- .ratio = 1,
- .htim = NULL, // Sẽ khởi tạo sau
- .channel = TIM_CHANNEL_2,
- .current_mode = MODE_STOP,
- .velocity = 0,
- .current_pulses = 0,
- .cnt = 0,
- .nbPulse = 0,
- .accPulse = 0,
- .decPulse = 0,
- .accSize = 0,
- .decSize = 0,
- .run = 0,
- .move = 0,
- .acc = 0,
- .stop = 0,
- .dec = 0,
- .softStop = 0,
- .arr = 0
- };
-
```

```

- uint8_t STEP4mode = 0; // Biến lưu chế độ cho động cơ bước (không sử dụng trong code này)
- uint8_t t4_1 = 0; // C? cho nút nhấn động cơ step1
- uint8_t t4_2 = 0; // C? cho nút nhấn động cơ step2
-
- MotorState currentState1 = STATE_STOP; // Trạng thái hiện tại của step1
- MotorState currentState2 = STATE_STOP; // Trạng thái hiện tại của step2
-
- uint32_t lastStepTime1 = 0; // Thời gian cuối cùng thực hiện bước cho step1
- uint32_t lastStepTime2 = 0; // Thời gian cuối cùng thực hiện bước cho step2
-
- uint32_t stepDelay1 = 1000; // Thời gian giữa các bước cho step1 (ms)
- uint32_t stepDelay2 = 1000; // Thời gian giữa các bước cho step2 (ms)
-
- // Khai báo mẫu chuyển động cho động cơ step1
- MotorPattern motor1Pattern = {
-     .mode = 0, // Bắt đầu với chế độ 0
-     .stepIndex = 0,
-     .stepAngles = {5, 5, 5, 5, 0}, // Mảng góc cho từng bước
-     .stepDirections = {1, 0, 0, 1, 0}, // Mảng hướng cho từng bước
-     .numSteps = 4 // Có 4 bước trong mẫu
- };
-
- // Khai báo mẫu chuyển động cho động cơ step2
- MotorPattern motor2Pattern = {
-     .mode = 0, // Bắt đầu với chế độ 0
-     .stepIndex = 0,
-     .stepAngles = {5, 5, 5, 5, 20}, // Mảng góc cho từng bước
-     .stepDirections = {0, 0, 0, 0, 1}, // Mảng hướng cho từng bước
-     .numSteps = 1 // Có 1 bước trong mẫu (sẽ được cập nhật tùy chế độ)
- };
-
- /* USER CODE END PV */
-
- /* Private function prototypes -----*/
- void SystemClock_Config(void);
- static void MX_GPIO_Init(void);
- static void MX_TIM2_Init(void);
- static void MX_TIM3_Init(void);
- /* USER CODE BEGIN PFP */
-
- // Khai báo các hàm xử lý chế độ và đi?u khiển động cơ
- void handle_STEP4mode1(void); // Xử lý chế độ cho step1

```

```

- void handle_STEP4mode2(void); // Xử lý chế độ cho step2
- void handle_motor(void); // Xử lý chung cho động cơ bước
- void handle_motor_step(StepMotor *motor, MotorPattern *pattern, MotorState
  *currentState, uint32_t *lastStepTimeVar, uint32_t stepDelay); // Xử lý từng bước cho
  động cơ bước
- void STEP_ProcessAll(void); // Xử lý các bước cho tất cả động cơ bước
-
- // Hàm đi?u khiển động cơ DC
- void Motor1_SetSpeed(uint16_t speed1);
- void Motor1_SetDirection(uint16_t direction1);
- void Motor2_SetSpeed(uint16_t speed2);
- void Motor2_SetDirection(uint16_t direction2);
- void Motor3_SetSpeed(uint16_t speed3);
- void SetMotorMode(void);
- void SetMotor3Mode(void);
-
- /* USER CODE END PFP */
-
- /* Private user code -----*/
- /* USER CODE BEGIN 0 */
-
- /* Hàm đi?u khiển động cơ DC */
-
- // Thiết lập tốc độ cho động cơ DC1
- void Motor1_SetSpeed(uint16_t speed1)
- {
-     if (speed1 > 100) speed1 = 100; // Giới hạn tốc độ tối đa 100%
-     uint32_t pulse1 = (htim2.Init.Period + 1) * speed1 / 100; // Tính giá trị PWM
-     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pulse1); // Thiết lập giá
  trị PWM cho kênh 1
- }
-
- // Thiết lập hướng cho động cơ DC1
- void Motor1_SetDirection(uint16_t direction1)
- {
-     if (direction1 == 1) // Quay thuận
-     {
-         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET); // IN1 = 1
-         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET); // IN2 = 0
-     }
-     else // Dừng hoặc quay ngược
-     {
-         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET); // IN1 = 0
-         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET); // IN2 = 0
-     }
- }

```

```
-     }  
- }  
-  
- // Thiết lập tốc độ cho động cơ DC2  
- void Motor2_SetSpeed(uint16_t speed2)  
- {  
-     if (speed2 > 100) speed2 = 100; // Giới hạn tốc độ tối đa 100%  
-     uint32_t pulse2 = (htim2.Init.Period + 1) * speed2 / 100; // Tính giá trị PWM  
-     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse2); // Thiết lập giá  
-     trị PWM cho kênh 2  
- }  
-  
- // Thiết lập hướng cho động cơ DC2  
- void Motor2_SetDirection(uint16_t direction2)  
- {  
-     if (direction2 == 2) // Quay thuận  
-     {  
-         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET); // IN1 = 1  
-         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET); // IN2 = 0  
-     }  
-     else // Dừng hoặc quay ngược  
-     {  
-         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // IN1 = 0  
-         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET); // IN2 = 0  
-     }  
- }  
-  
- // Thiết lập tốc độ cho động cơ DC3  
- void Motor3_SetSpeed(uint16_t speed3)  
- {  
-     if (speed3 > 100) speed3 = 100; // Giới hạn tốc độ tối đa 100%  
-     uint32_t pulse3 = (htim2.Init.Period + 1) * speed3 / 100; // Tính giá trị PWM  
-     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, pulse3); // Thiết lập giá  
-     trị PWM cho kênh 3  
- }  
-  
- // Thiết lập chế độ cho động cơ DC1 và DC2  
- void SetMotorMode(void)  
- {  
-     switch (DCmode)  
-     {  
-     case 1:  
-         // Chế độ 1: Hai động cơ chạy bằng nhau  
-         Motor1_SetSpeed(10);
```

```
- Motor1_SetDirection(1);
- Motor2_SetSpeed(10);
- Motor2_SetDirection(2);
- break;
- case 2:
- // Chế độ 2: Động cơ 1 chạy nhanh hơn một chút
- Motor1_SetSpeed(15);
- Motor1_SetDirection(1);
- Motor2_SetSpeed(10);
- Motor2_SetDirection(2);
- break;
- case 3:
- // Chế độ 3: Động cơ 1 chạy nhanh hơn nhì
- Motor1_SetSpeed(15);
- Motor1_SetDirection(1);
- Motor2_SetSpeed(15);
- Motor2_SetDirection(2);
- break;
- case 4:
- // Chế độ 4: Cả hai động cơ chạy chậm
- Motor1_SetSpeed(20);
- Motor1_SetDirection(1);
- Motor2_SetSpeed(10);
- Motor2_SetDirection(2);
- break;
- default:
- // Dừng cả hai động cơ
- Motor1_SetSpeed(0);
- Motor1_SetDirection(0);
- Motor2_SetSpeed(0);
- Motor2_SetDirection(0);
- break;
- }
- }
-
- // Thiết lập chế độ cho động cơ DC3
- void SetMotor3Mode(void)
- {
- switch (DCmode1)
- {
- case 1:
- Motor3_SetSpeed(18);
- break;
- case 2:
```

```
-     Motor3_SetSpeed(24);
-     break;
-     case 3:
-         Motor3_SetSpeed(28);
-         break;
-     case 4:
-         Motor3_SetSpeed(15);
-         break;
-     default:
-         Motor3_SetSpeed(0);
-         break;
- }
- }
-
- /* Hàm đi u khiển động cơ bước */
-
- // Hàm xử lý chế độ cho động cơ bước 1 khi nhấn nút
- void handle_STEP4mode1(void)
- {
-     // Tăng chế độ
-     motor1Pattern.mode++;
-     if (motor1Pattern.mode > 3)
-     {
-         motor1Pattern.mode = 0;
-     }
-
-     // Thiết lập mẫu chuyển động dựa trên chế độ
-     switch (motor1Pattern.mode)
-     {
-     case 1:
-         // Chế độ 1: Quay 4 bước với góc 5 độ
-         motor1Pattern.stepAngles[0] = 5;
-         motor1Pattern.stepDirections[0] = 1;
-
-         motor1Pattern.stepAngles[1] = 5;
-         motor1Pattern.stepDirections[1] = 0;
-
-         motor1Pattern.stepAngles[2] = 5;
-         motor1Pattern.stepDirections[2] = 0;
-
-         motor1Pattern.stepAngles[3] = 5;
-         motor1Pattern.stepDirections[3] = 1;
-
-         motor1Pattern.numSteps = 4;
```



```
-         break;
-
-     case 2:
-         // Chế độ 2: Quay 4 bước với góc 10 độ
-         motor1Pattern.stepAngles[0] = 10;
-         motor1Pattern.stepDirections[0] = 1;
-
-         motor1Pattern.stepAngles[1] = 10;
-         motor1Pattern.stepDirections[1] = 0;
-
-         motor1Pattern.stepAngles[2] = 10;
-         motor1Pattern.stepDirections[2] = 0;
-
-         motor1Pattern.stepAngles[3] = 10;
-         motor1Pattern.stepDirections[3] = 1;
-
-         motor1Pattern.numSteps = 4;
-         break;
-
-     case 3:
-         // Chế độ 3: Quay 4 bước với góc 20 độ
-         motor1Pattern.stepAngles[0] = 20;
-         motor1Pattern.stepDirections[0] = 1;
-
-         motor1Pattern.stepAngles[1] = 20;
-         motor1Pattern.stepDirections[1] = 0;
-
-         motor1Pattern.stepAngles[2] = 20;
-         motor1Pattern.stepDirections[2] = 0;
-
-         motor1Pattern.stepAngles[3] = 20;
-         motor1Pattern.stepDirections[3] = 1;
-
-         motor1Pattern.numSteps = 4;
-         break;
-
-     default:
-         // Dừng động cơ nếu chế độ không hợp lệ
-         STEP_HardStop(&step1);
-         motor1Pattern.numSteps = 0;
-         currentState1 = STATE_STOP;
-         break;
- }
-
```

```
- // Reset chỉ số bước và trạng thái
- motor1Pattern.stepIndex = 0;
- if(motor1Pattern.numSteps > 0){
-     currentState1 = STATE_EXECUTE_STEP_SEQUENCE;
- } else {
-     currentState1 = STATE_STOP;
- }
- }
-
- // Hàm xử lý chế độ cho động cơ bước 2 khi nhấn nút
- void handle_STEP4mode2(void)
- {
-     // Tăng chế độ
-     motor2Pattern.mode++;
-     if (motor2Pattern.mode > 5)
-     {
-         motor2Pattern.mode = 1;
-     }
-
-     // Thiết lập mẫu chuyển động dựa trên chế độ
-     switch (motor2Pattern.mode)
-     {
-     case 1:
-     case 2:
-     case 3:
-     case 4:
-         // Chế độ 1-4: Quay 1 bước với góc 5 độ
-         motor2Pattern.stepAngles[0] = 5;
-         motor2Pattern.stepDirections[0] = 0;
-         motor2Pattern.numSteps = 1;
-         break;
-
-     case 5:
-         // Chế độ 5: Quay 1 bước với góc 20 độ
-         motor2Pattern.stepAngles[0] = 20;
-         motor2Pattern.stepDirections[0] = 1;
-         motor2Pattern.numSteps = 1;
-         break;
-
-     default:
-         // Dừng động cơ nếu chế độ không hợp lệ
-         STEP_HardStop(&step2);
-         motor2Pattern.numSteps = 0;
-         currentState2 = STATE_STOP;
```

```

-         break;
-     }
-
-     // Reset chỉ số bước và trạng thái
-     motor2Pattern.stepIndex = 0;
-     if(motor2Pattern.numSteps > 0){
-         currentState2 = STATE_EXECUTE_STEP_SEQUENCE;
-     } else {
-         currentState2 = STATE_STOP;
-     }
- }
-
- // Hàm xử lý từng bước cho một động cơ bước
- void handle_motor_step(StepMotor *motor, MotorPattern *pattern, MotorState
*currentState, uint32_t *lastStepTimeVar, uint32_t stepDelay)
- {
-     uint32_t currentTime = HAL_GetTick(); // Lấy thời gian hiện tại
-
-     if (*currentState == STATE_EXECUTE_STEP_SEQUENCE)
-     {
-         // Kiểm tra thời gian để thực hiện bước tiếp theo
-         if (currentTime - (*lastStepTimeVar) >= stepDelay)
-         {
-             (*lastStepTimeVar) = currentTime;
-
-             // Lấy góc và hướng di chuyển cho bước hiện tại
-             uint32_t angle = pattern->stepAngles[pattern->stepIndex];
-             uint32_t pulses = STEPMOTOR_STEPS * angle / 360; // Tính số xung cần để
quay góc này
-             uint8_t direction = pattern->stepDirections[pattern->stepIndex];
-
-             // Thiết lập hướng quay
-             STEP_SetDirection(motor, direction);
-
-             // Thiết lập tốc độ quay (có thể điều chỉnh)
-             STEP_SetSpeed(motor, 5);
-
-             // Thực hiện di chuyển
-             STEP_Move(motor, pulses);
-
-             // Cập nhật chỉ số bước
-             pattern->stepIndex++;
-             if (pattern->stepIndex >= pattern->numSteps)
-             {

```

```

-         pattern->stepIndex = 0;
-
-         // Nếu là động cơ step2, dừng sau khi hoàn thành
-         if (motor == &step2)
-         {
-             *currentState = STATE_STOP;
-         }
-     }
- }
-
- else
- {
-     // Dừng động cơ nếu không trong trạng thái di chuyển
-     STEP_HardStop(motor);
- }
- }
-
- // Hàm xử lý động cơ bước (gọi cho cả hai động cơ)
- void handle_motor(void)
- {
-     handle_motor_step(&step1, &motor1Pattern, &currentState1, &lastStepTime1,
- stepDelay1);
-     handle_motor_step(&step2, &motor2Pattern, &currentState2, &lastStepTime2,
- stepDelay2);
- }
-
- // Hàm xử lý các bước di chuyển cho tất cả động cơ bước
- void STEP_ProcessAll(void)
- {
-     STEP_Process(&step1);
-     STEP_Process(&step2);
- }
-
- /* USER CODE END 0 */
-
- /**
-  * @brief The application entry point.
-  * @retval int
-  */
- int main(void)
- {
-
-     /* USER CODE BEGIN 1 */
-     // (Nếu bạn có khai báo biến toàn cục mới, thêm vào đây)

```

```
- /* USER CODE END 1 */
-
- /* MCU Configuration-----*/
-
- /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
- HAL_Init();
-
- /* USER CODE BEGIN Init */
- // (Nếu bạn cần khởi tạo gì thêm, thêm vào đây)
- /* USER CODE END Init */
-
- /* Configure the system clock */
- SystemClock_Config();
-
- /* USER CODE BEGIN SysInit */
- // (Nếu bạn cần cấu hình gì thêm, thêm vào đây)
- /* USER CODE END SysInit */
-
- /* Initialize all configured peripherals */
- MX_GPIO_Init();
- MX_TIM2_Init();
- MX_TIM3_Init();
- /* USER CODE BEGIN 2 */
-
- /* Khởi tạo động cơ bước */
- step1.htim = &htim3; // Gán timer cho step1
- step2.htim = &htim3; // Gán timer cho step2
-
- STEP_Init(&step1); // Khởi tạo step1
- STEP_HardStop(&step1); // Dừng step1
-
- STEP_Init(&step2); // Khởi tạo step2
- STEP_HardStop(&step2); // Dừng step2
-
- /* Khởi động PWM cho các kênh đi  u khiển động cơ DC */
- HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1); // DC Motor 1
- HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2); // DC Motor 2
- HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3); // DC Motor 3
-
- /* Khởi tạo tốc độ và hướng ban đầu cho động cơ DC */
- Motor1_SetSpeed(0);
- Motor1_SetDirection(0);
- Motor2_SetSpeed(0);
- Motor2_SetDirection(0);
```

```
- Motor3_SetSpeed(0);
-
- /* USER CODE END 2 */
-
- /* Infinite loop */
- /* USER CODE BEGIN WHILE */
- while (1)
- {
-     // Kiểm tra và xử lý nút nhấn cho động cơ step1
-     if (t4_1 == 1)
-     {
-         handle_STEP4mode1();
-         t4_1 = 0;
-     }
-
-     // Kiểm tra và xử lý nút nhấn cho động cơ step2
-     if (t4_2 == 1)
-     {
-         handle_STEP4mode2();
-         t4_2 = 0;
-     }
-
-     // Kiểm tra và xử lý nút nhấn cho động cơ DC1 và DC2
-     if (t == 1)
-     {
-         SetMotorMode();
-         t = 0;
-     }
-
-     // Kiểm tra và xử lý nút nhấn cho động cơ DC3
-     if (t1 == 1)
-     {
-         SetMotor3Mode();
-         t1 = 0;
-     }
-
-     // Xử lý động cơ bước nếu cần
-     if (currentState1 != STATE_STOP || currentState2 != STATE_STOP)
-     {
-         handle_motor();
-     }
-
-     // Xử lý các bước di chuyển cho động cơ bước
-     STEP_ProcessAll();
```

```
-
- HAL_Delay(10); // Giảm tải CPU
-
- /* USER CODE END WHILE */
-
- /* USER CODE BEGIN 3 */
- // (Nếu bạn có code gì thêm, thêm vào đây)
- }
- /* USER CODE END 3 */
- }
-
- /**
-  * @brief System Clock Configuration
-  * @retval None
-  */
- void SystemClock_Config(void)
- {
-     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
-     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
-
-     /** Initializes the RCC Oscillators according to the specified parameters
-      * in the RCC_OscInitTypeDef structure.
-      */
-     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
-     RCC_OscInitStruct.HSEState = RCC_HSE_ON;
-     RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
-     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
-     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
-     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
-     RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
-     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
-     {
-         Error_Handler();
-     }
-
-     /** Initializes the CPU, AHB and APB buses clocks
-      */
-     RCC_ClkInitStruct.ClockType =
- RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
- |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
-     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
-     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
-     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
-     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
```

```
-
-  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
-  HAL_OK)
-  {
-      Error_Handler();
-  }
-
-  /**
-   * @brief TIM2 Initialization Function
-   * @param None
-   * @retval None
-   */
-  static void MX_TIM2_Init(void)
-  {
-
-      /* USER CODE BEGIN TIM2_Init 0 */
-      // Timer 2 được sử dụng cho PWM để ?u khiển động cơ DC
-      /* USER CODE END TIM2_Init 0 */
-
-      TIM_ClockConfigTypeDef sClockSourceConfig = {0};
-      TIM_MasterConfigTypeDef sMasterConfig = {0};
-      TIM_OC_InitTypeDef sConfigOC = {0};
-
-      /* USER CODE BEGIN TIM2_Init 1 */
-      // Không có gì cần thêm ở đây
-      /* USER CODE END TIM2_Init 1 */
-      htim2.Instance = TIM2;
-      htim2.Init.Prescaler = 71;
-      htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
-      htim2.Init.Period = 999;
-      htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
-      htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
-      if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
-      {
-          Error_Handler();
-      }
-      sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
-      if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
-      {
-          Error_Handler();
-      }
-      if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
-      {
-          Error_Handler();
-      }
-  }
```



```
- Error_Handler();
- }
- sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
- sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
- if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
  HAL_OK)
- {
-   Error_Handler();
- }
- sConfigOC.OCMode = TIM_OCMODE_PWM1;
- sConfigOC.Pulse = 0;
- sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
- sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
- if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
  HAL_OK)
- {
-   Error_Handler();
- }
- if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_2) !=
  HAL_OK)
- {
-   Error_Handler();
- }
- if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_3) !=
  HAL_OK)
- {
-   Error_Handler();
- }
- /* USER CODE BEGIN TIM2_Init 2 */
- // Không có gì cần thêm ở đây
- /* USER CODE END TIM2_Init 2 */
- HAL_TIM_MspPostInit(&htim2);
-
- }
-
- /**
-  * @brief TIM3 Initialization Function
-  * @param None
-  * @retval None
-  */
- static void MX_TIM3_Init(void)
- {
-
-
-   /* USER CODE BEGIN TIM3_Init 0 */
```

```
- // Timer 3 được sử dụng cho đi?u khiển động cơ bước
- /* USER CODE END TIM3_Init 0 */
-
- TIM_MasterConfigTypeDef sMasterConfig = {0};
- TIM_OC_InitTypeDef sConfigOC = {0};
-
- /* USER CODE BEGIN TIM3_Init 1 */
- // Không có gì cần thêm ở đây
- /* USER CODE END TIM3_Init 1 */
- htim3.Instance = TIM3;
- htim3.Init.Prescaler = 7;
- htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
- htim3.Init.Period = 4999;
- htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
- htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
- if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
- {
-     Error_Handler();
- }
- sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
- sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
- if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) !=
    HAL_OK)
- {
-     Error_Handler();
- }
- sConfigOC.OCMode = TIM_OCMODE_PWM1;
- sConfigOC.Pulse = 0;
- sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
- sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
- if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1) !=
    HAL_OK)
- {
-     Error_Handler();
- }
- if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2) !=
    HAL_OK)
- {
-     Error_Handler();
- }
- /* USER CODE BEGIN TIM3_Init 2 */
- // Không có gì cần thêm ở đây
- /* USER CODE END TIM3_Init 2 */
- HAL_TIM_MspPostInit(&htim3);
```

```
-  
- }  
-  
- /**  
-  * @brief GPIO Initialization Function  
-  * @param None  
-  * @retval None  
-  */  
- static void MX_GPIO_Init(void)  
- {  
-     GPIO_InitTypeDef GPIO_InitStruct = {0};  
-     /* USER CODE BEGIN MX_GPIO_Init_1 */  
-     /* USER CODE END MX_GPIO_Init_1 */  
-  
-     /* GPIO Ports Clock Enable */  
-     __HAL_RCC_GPIOD_CLK_ENABLE();  
-     __HAL_RCC_GPIOA_CLK_ENABLE();  
-     __HAL_RCC_GPIOB_CLK_ENABLE();  
-  
-     /*Configure GPIO pin Output Level */  
-     HAL_GPIO_WritePin(GPIOA, In1_Pin|IN2_Pin, GPIO_PIN_RESET);  
-  
-     /*Configure GPIO pin Output Level */  
-     HAL_GPIO_WritePin(GPIOB, IN1_Pin|IN2B1_Pin|GPIO_PIN_12|GPIO_PIN_13  
-         |GPIO_PIN_14|GPIO_PIN_15, GPIO_PIN_RESET);  
-  
-     /*Configure GPIO pins : In1_Pin IN2_Pin */  
-     GPIO_InitStruct.Pin = In1_Pin|IN2_Pin;  
-     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;  
-     GPIO_InitStruct.Pull = GPIO_PULLUP;  
-     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;  
-     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);  
-  
-     /*Configure GPIO pins : PA5 PA8 PA9 */  
-     GPIO_InitStruct.Pin = GPIO_PIN_5|GPIO_PIN_8|GPIO_PIN_9;  
-     GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;  
-     GPIO_InitStruct.Pull = GPIO_PULLUP;  
-     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);  
-  
-     /*Configure GPIO pins : IN1_Pin IN2B1_Pin */  
-     GPIO_InitStruct.Pin = IN1_Pin|IN2B1_Pin;  
-     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;  
-     GPIO_InitStruct.Pull = GPIO_PULLUP;  
-     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

```
- HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
-
- /*Configure GPIO pins : PB12 PB13 PB14 PB15 */
- GPIO_InitStruct.Pin = GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15;
- GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
- GPIO_InitStruct.Pull = GPIO_NOPULL;
- GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
- HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
-
- /*Configure GPIO pin : PB7 */
- GPIO_InitStruct.Pin = GPIO_PIN_7;
- GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
- GPIO_InitStruct.Pull = GPIO_PULLUP;
- HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
-
- /* EXTI interrupt init*/
- HAL_NVIC_SetPriority(EXTI9_5_IRQn, 1, 0);
- HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
-
- /* USER CODE BEGIN MX_GPIO_Init_2 */
- /* USER CODE END MX_GPIO_Init_2 */
- }
-
- /* USER CODE BEGIN 4 */
-
- // Hàm xử lý ngắt EXTI cho nút nhấn
- void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
- {
-     // Biến tĩnh để lưu thời gian và bộ đếm debounce cho từng nút
-     static uint32_t last_interrupt_time_step1 = 0;
-     static uint32_t last_interrupt_time_step2 = 0;
-     static uint32_t last_interrupt_time_dc1dc2 = 0;
-     static uint32_t last_interrupt_time_dc3 = 0;
-
-     static uint8_t debounce_counter_step1 = 0;
-     static uint8_t debounce_counter_step2 = 0;
-     static uint8_t debounce_counter_dc1dc2 = 0;
-     static uint8_t debounce_counter_dc3 = 0;
-
-     uint32_t current_time = HAL_GetTick();
-
-     switch(GPIO_Pin)
-     {
-     case GPIO_PIN_8: // Nút nhấn cho động cơ step1
-         if ((current_time - last_interrupt_time_step1) > DEBOUNCE_DELAY)
-         {
-             // Kiểm tra trạng thái nút (giả sử nút là active low)
-             if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET)
```

```

-         {
-             debounce_counter_step1++;
-             if (debounce_counter_step1 >= DEBOUNCE_COUNT)
-             {
-                 t4_1 = 1; // Đặt cờ để xử lý trong vòng lặp chính
-                 HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); // Bật/tắt LED
-                 debounce_counter_step1 = 0; // Reset bộ đếm
-             }
-         }
-     else
-     {
-         debounce_counter_step1 = 0; // Reset bộ đếm nếu nút không còn được nhấn
-     }
-     last_interrupt_time_step1 = current_time; // Cập nhật thời gian ngắt cuối cùng
- }
- break;
-
- case GPIO_PIN_9: // Nút nhấn cho động cơ step2
-     if ((current_time - last_interrupt_time_step2) > DEBOUNCE_DELAY)
-     {
-         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_RESET)
-         {
-             debounce_counter_step2++;
-             if (debounce_counter_step2 >= DEBOUNCE_COUNT)
-             {
-                 t4_2 = 1;
-                 HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
-                 debounce_counter_step2 = 0;
-             }
-         }
-     else
-     {
-         debounce_counter_step2 = 0;
-     }
-     last_interrupt_time_step2 = current_time;
- }
- break;
-
- case GPIO_PIN_5: // Nút nhấn cho động cơ DC1 và DC2
-     if ((current_time - last_interrupt_time_dc1dc2) > DEBOUNCE_DELAY)
-     {
-         if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_5) == GPIO_PIN_RESET)
-         {
-             debounce_counter_dc1dc2++;
-             if (debounce_counter_dc1dc2 >= DEBOUNCE_COUNT)
-             {
-                 DCmode++;
-                 if (DCmode > 3)
-                 {
-                     DCmode = 0;
-                 }
-                 t = 1;
-                 HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);

```

```

-         debounce_counter_dc1dc2 = 0;
-     }
- }
- else
- {
-     debounce_counter_dc1dc2 = 0;
- }
- last_interrupt_time_dc1dc2 = current_time;
- }
- break;
-
- case GPIO_PIN_7: // Nút nhấn cho động cơ DC3
-     if ((current_time - last_interrupt_time_dc3) > DEBOUNCE_DELAY)
-     {
-         if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_7) == GPIO_PIN_RESET)
-         {
-             debounce_counter_dc3++;
-             if (debounce_counter_dc3 >= DEBOUNCE_COUNT)
-             {
-                 DCmode1++;
-                 if (DCmode1 > 3)
-                 {
-                     DCmode1 = 0;
-                 }
-                 t1 = 1;
-                 HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
-                 debounce_counter_dc3 = 0;
-             }
-         }
-         else
-         {
-             debounce_counter_dc3 = 0;
-         }
-         last_interrupt_time_dc3 = current_time;
-     }
-     break;
-
- default:
-     // Không làm gì nếu không phải các nút đã định nghĩa
-     break;
- }
- }
-
- // Hàm xử lý ngắt Timer cho động cơ bước (nếu cần)
- void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
- {
-     if (htim->Instance == TIM3)
-     {
-         STEP_ProcessAll(); // Xử lý các bước cho động cơ bước
-     }
- }

```

```
-
- /* USER CODE END 4 */
-
- /**
-  * @brief This function is executed in case of error occurrence.
-  * @retval None
-  */
- void Error_Handler(void)
- {
-  /* USER CODE BEGIN Error_Handler_Debug */
-
-  __disable_irq();
-  while (1)
-  {
-  }
-  /* USER CODE END Error_Handler_Debug */
- }
-
- #ifdef USE_FULL_ASSERT
- /**
-  * @brief Reports the name of the source file and the source line number
-  *        where the assert_param error has occurred.
-  * @param file: pointer to the source file name
-  * @param line: assert_param error line source number
-  * @retval None
-  */
- void assert_failed(uint8_t *file, uint32_t line)
- {
-  /* USER CODE BEGIN 6 */
-
-  /* USER CODE END 6 */
- }
- #endif /* USE_FULL_ASSERT */
-
```

CHAPTER 9: CONCLUSION AND DEVELOPMENT DIRECTION

9.1 Improving Mechanical Design

The mechanical design plays a crucial role in ensuring the stability, accuracy, and durability of the system. The following improvements can be made to enhance the overall performance:

- **Optimize Structural Components:** Redesign key mechanical parts to minimize vibrations during operation, which can lead to inaccuracies in ball trajectories and unnecessary wear on the system.
- **Material Selection:** Use lightweight yet strong materials such as aluminum alloys or carbon fiber to reduce the load on motors, improve energy efficiency, and increase portability without compromising structural integrity.
- **Simplified Assembly and Maintenance:** Streamline the mechanical design to make the system easier to assemble and disassemble, reducing maintenance time and costs.
- **Compact Design:** Improve the compactness of the machine to save space and make it more suitable for smaller environments, such as home use.
- **Enhance Durability:** Reinforce critical components, such as the launching mechanism and motor mounts, to withstand long-term use and high-speed operations.

9.2 Using Appropriate Frequencies to Reduce Motor Noise

Motor noise can significantly affect the user experience, especially in indoor environments. Adjusting the operating frequencies and improving the motor control system can help address this issue:

- **Optimize PWM Frequency:** By selecting a PWM frequency that is above the human hearing range (typically >20 kHz), the audible motor noise can be minimized while maintaining precise speed control. This adjustment ensures smoother and quieter motor operation.
- **Noise-Reducing Motor Drivers:** Use advanced motor drivers with built-in noise reduction features or soft-start functionality to eliminate sudden noise spikes during operation.
- **Vibration Dampening:** Incorporate rubber mounts or other dampening materials between the motor and frame to absorb vibrations that may contribute to noise.
- **Improved Cooling System:** If the motor produces noise due to overheating, adding an efficient cooling system such as heat sinks or fans can reduce thermal strain, indirectly lowering noise levels.

9.3 Expanding Operational Modes

Adding multiple operational modes enhances the versatility and adaptability of the system, making it suitable for various training scenarios and user preferences. The following modes can be implemented:

- Training Mode:
 - Program repetitive launches with fixed angles, velocities, and spin types to allow users to practice specific skills, such as consistent forehand or backhand strokes.
 - Enable speed and frequency adjustments to suit beginners, intermediate players, or professionals.
- Random Mode:
 - Introduce randomness in ball trajectories, such as varying angles, speeds, and spins, to simulate real-game conditions and improve reflexes.
 - Use an algorithm to generate unpredictable patterns for more engaging training sessions.
- Custom Mode:
 - Allow users to manually set and save parameters, including launch angles, spin direction, and speed, for tailored training sessions.
 - Include an option to schedule different sequences of ball launches to simulate real-world scenarios, such as returning multiple serves.
- Interactive or Feedback-Based Mode:
 - Integrate sensors to track the user's performance and automatically adjust parameters based on their input or success rate.
 - For instance, if the player consistently misses the ball at certain speeds, the system can adapt to provide more manageable challenges.
- Remote Control Mode:
 - Incorporate wireless connectivity (e.g., Bluetooth or Wi-Fi) to allow users to control the machine remotely via a mobile app.
 - Include features such as parameter adjustment, mode selection, and real-time feedback display in the app.

Table of Figure:

Figure 2.1: Table tennis competition at the 2012 Olympics.....	6
Figure 2.2: Standard table tennis table size.....	7
Figure 2.3: Practice with the table tennis machine.....	8
Figure 2.4: Robo Pong 540 with compact design.....	9
Figure 2.5: Hiepasc table tennis machine.....	10
Figure 3.1 High flight range.....	16
Figure 3.2 Long range.....	17
Figure 3.3 Ball Launch.....	18
Figure 4.1 Contact pattern between two wheels.....	20
Figure 4.2 RS-775 DC Motor.....	21
Figure 4.3 GA12-N20 motor.....	22
Figure 4.4 RS385 DC motor.....	26
Figure 4.5 Step motor 57HS56.....	29
Figure 4.6 Step motor 57HPB.....	31
Figure 5.1 Complete model	33
Figure 5.2 Ball leveling mechanism	34
Figure 5.3 Ball push mechanism.....	35
Figure 5.4 Ball shooting mechanism.....	35
Figure 5.5 Real-Life Model.....	46
Figure 6.1 Connect System	47
Figure 6.2 DCMotorDriverCircuit-XY-160D	48
Figure 6.3 L298 DC Motor Driver.....	49
Figure 6.4 Mini PWM DC Motor Speed Controller.....	51
Figure 6.5 Driver TB6600.....	52
Figure 7.1 Example Results from Simulation	54
Figure 8.1 STM32F103C8T6 Microcontroller	73

REFERENCES

- [1] <https://loigiaiway.com/ly-thuyet-chuyen-dong-nem-vat-li-10-a109786.html>
[2] <https://www.ijltet.org/wp-content/uploads/2014/05/24.pdf>